POCKET KNIFE – Manual

Inventory and software deployment for Windows computers in an Active Directory domain

Version 2.29.5 · Date 2026-10-24

Contents

Overview

Installation and Update
- Domain Prerequisites
- Set up and secure WinRM by Group Policy
- System requirements and sizing
- Installation
- Update
- First Steps

Using the Interface
- Navigation and Language
- News and updates
- Tables
- Keyboard Shortcuts

Computers and Collections
- Computer List
- Collections
- AD Collections
- Computer View

Scan Profiles and Scanners
- Scan Profiles
- Custom Scanners

Software

Packages
- Step Types
- Conditions
- Run as
- Examples
- Proxy server
- winget on Servers

Repository
- Restructure the repository
- Duplicate block
- Find file (repair by fingerprint)

Deployments
- Notification on the client
- Message on restart and shutdown
- POCKET KNIFE restarted during a deployment

Schedules

Tools
- Files (Explorer)
- Event log

Software Kiosk
- Sign-in with Windows (SSO)

Remote Sessions

Reports and Log
- Custom reports
- Reports by e-mail
- Notifications
- Setting up the SMTP server

Settings
- OU admins (tenant admins)
- Your own certificate
- Kerberos instead of NTLM
- Diagnostics package

Security and Backup

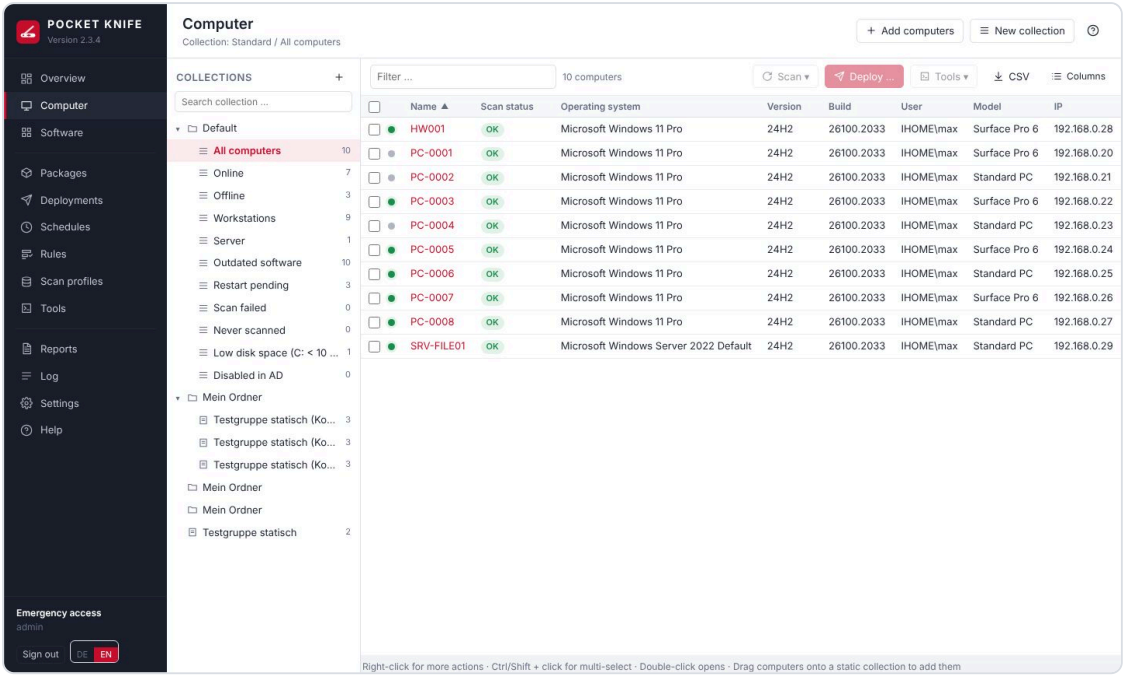
Troubleshooting

Glossary

Overview

POCKET KNIFE is a web application that inventories the Windows computers in an Active Directory domain (Microsoft AD or Samba AD) and installs, updates and removes software on them. It runs as a Docker container; all data is stored in a SQLite database in the data directory `/data`. The interface loads nothing from the internet – no fonts, scripts or images from third-party servers.

Website: helfi.space



Computer list with collection tree

How POCKET KNIFE works:

- **Computers** are imported automatically from AD (sync over LDAPS).
- **Scan profiles** use WinRM to read what is installed on the computers and what hardware they have.
- **Collections** group computers – statically by hand or dynamically through filters (e.g. “Firefox outdated”).
- **Packages** describe what runs on a computer: installations, commands, scripts, restarts …
- **Deployments** run a package on computers – immediately, at a set time, through **schedules** (packages or winget updates).

- **Tools** are quick one-off actions (ping, restart, gpupdate, custom scripts).

Terms

Term	Meaning
Service account	Domain account with local administrator rights on the clients. POCKET KNIFE uses it for LDAPS, WinRM and SMB.
Repository	Network share (or mounted folder) containing your installation files.
winget	Microsoft's package manager; POCKET KNIFE matches installed software against the winget catalog and can install and update software through it.
Heartbeat	Regular check of which computers are reachable.
Tenant	Optional separation of packages and schedules, e.g. by customer or site.

Installation and Update

Domain Prerequisites

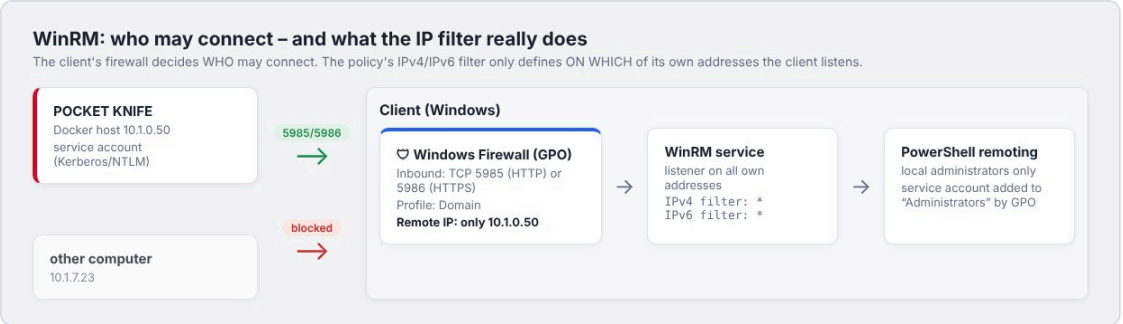
1. Create a **service account**, e.g. `svc-pocketknife` (a regular user account whose password never expires).
2. Use a GPO to add the service account to the local **Administrators** group on all clients (*Computer Configuration → Preferences → Control Panel Settings → Local Users and Groups*).
3. Enable and secure **WinRM** through a GPO – step by step in WinRM by Group Policy.
4. In the client firewall, allow **TCP 5985** (or 5986 for HTTPS) and **TCP 445 only from the Docker host**. For remote sessions, also allow **TCP 3389** (RDP) or **5900** (VNC).
5. Create the AD group **POCKETKNIFE-Admins** and add the operators (nested groups are supported).
6. **LDAPS** must be enabled on the domain controllers. A self-signed certificate is sufficient – POCKET KNIFE trusts it automatically and logs every change of the fingerprint.
7. For custom packages, create a share, e.g. `\\fileserver\software$`, with read access for the service account.

Set up and secure WinRM by Group Policy

POCKET KNIFE works on the clients through **WinRM** (Windows Remote Management, PowerShell remoting): inventory, deployments, tools, file explorer and event viewer all use it. Nothing is installed on the client – which means WinRM has to be set up the same way and **securely** everywhere. The simplest and most traceable way is **a dedicated Group Policy** (e.g. “WinRM-Clients”) linked to the OUs of the computers POCKET KNIFE should manage. This guide applies to Windows 10/11 and Windows Server 2016 and later; the names are those of the English Group Policy Management Editor.

> **Rule of thumb:** WinRM is only as secure as its settings. The policy makes sure that **only the POCKET KNIFE server** reaches port 5985/5986, that traffic is **always encrypted** and **only domain**

> **sign-in** (Kerberos/NTLM) is used, and that nobody weakens the settings locally.

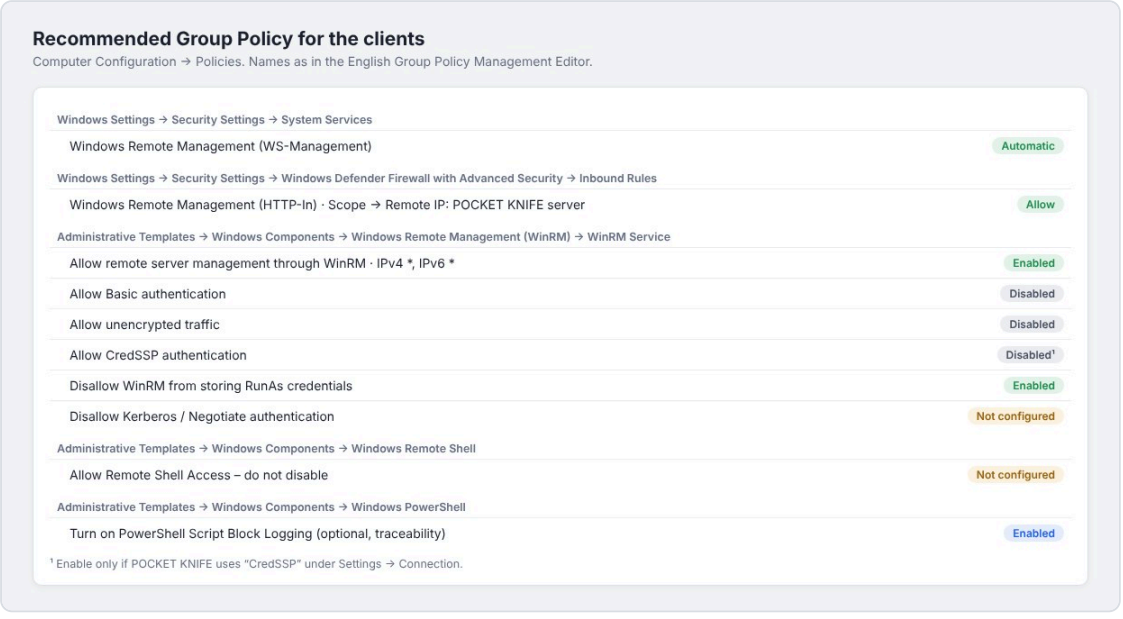


WinRM: firewall and IP filter

Step 1: Create the Group Policy

1. Open **Group Policy Management** (`gpmc.msc`) → right-click the OU with the clients → **Create a GPO in this domain, and Link it here** → name e.g. `WinRM-Clients`.
2. Use a separate OU or policy for servers (different maintenance windows, WinRM often already enabled). Include **domain controllers** only deliberately and with a policy of their own.
3. For testing, link a small OU with a few computers first, check it (step 8), then roll it out.

All of the following settings are under **Computer Configuration → Policies**.



Recommended Group Policy

Step 2: Start the service automatically

Windows Settings → Security Settings → System Services → ***Windows Remote Management (WS-Management)*** → "Define this policy setting" → **Automatic**. Do not change the permissions.

Step 3: Enable the WinRM listener

Administrative Templates → Windows Components → Windows Remote Management (WinRM) → WinRM Service → ***Allow remote server management through WinRM*** → **Enabled**, IPv4 filter `*`, IPv6 filter `*`.

Important – a common misunderstanding: the IPv4/IPv6 filter defines **on which of the client's own addresses** WinRM listens – **not** which computers may connect. `*` is therefore correct (otherwise WinRM stops working after a DHCP address change). Who may connect is controlled by the firewall alone (step 4). An empty filter means: WinRM listens nowhere.

Step 4: Firewall – only the POCKET KNIFE server may connect

Windows Settings → Security Settings → ***Windows Defender Firewall with Advanced Security*** *→ Inbound Rules →* **New Rule**:

1. **Predefined** → "Windows Remote Management" → choose the rule **Windows Remote Management (HTTP-In)** (for HTTPS a **Port** rule for TCP 5986 instead) → **Allow the connection**.
2. Then open the rule → tab **Scope** → **Remote IP address: These IP addresses** → enter the address of POCKET KNIFE's **Docker host** (all of them with several instances). The clients see the Docker host's address, not the container's.
3. Tab **Advanced** → profile **Domain** only.

The same restriction is recommended for **TCP 445** (SMB: package copies via `ADMIN$`, file explorer) and – only if used – for **TCP 3389** (RDP) and **5900** (VNC). To prevent local administrators from opening ports themselves, set "Apply local firewall rules" to **No** under *Windows Defender Firewall Properties → Domain Profile → Customize settings* – check first whether applications need rules of their own.

Step 5: Harden the WinRM service

In the same folder as step 3 (*WinRM Service*):

Setting	Value	Why
Allow Basic authentication	**Disabled**	Basic sends user and password only Base64-encoded – POCKET KNIFE never needs it
Allow unencrypted traffic	**Disabled**	POCKET KNIFE encrypts also over HTTP 5985 (Kerberos/NTLM message encryption); unencrypted would only be a risk
Allow CredSSP authentication	**Disabled** – POCKET KNIFE does not use CredSSP	CredSSP passes the credentials on to the client; an attacker with administrator rights on the client could grab them
Disallow WinRM from storing RunAs credentials	**Enabled**	no stored passwords in WinRM plug-ins
Disallow Kerberos authentication / Disallow Negotiate authentication	**Not configured**	POCKET KNIFE signs in with these – enabling them locks it out
Turn On Compatibility HTTP/HTTPS Listener	**Not configured**	legacy ports 80/443, not needed

Under *Windows Components → Windows Remote Shell* do **not disable Allow Remote Shell Access**: it also blocks PowerShell remoting – POCKET KNIFE can then no longer reach the clients.

The *WinRM Client* folder only affects connections that **originate** from the client (e.g. an admin PC with `Enter-PSSession`). It does not matter for POCKET KNIFE; "Disallow Digest authentication" and "Allow Basic authentication: Disabled" do no harm there and are good hardening. Do not set "Trusted Hosts" to `*` – that turns off verification of the remote side with NTLM.

Step 6: Who may sign in?

By default WinRM only admits **local administrators** (and the group "Remote Management Users"). POCKET KNIFE needs administrator rights; the service account is added to the local group **Administrators** by Group Policy (see prerequisites), preferably through a domain group, e.g. `PK-Clientadmins`.

- Do **not** set `LocalAccountTokenFilterPolicy`: it is only needed for local accounts and weakens UAC protection for remote access.

- Do not change WinRM's permissions (SDDL) by policy – the firewall (step 4) and the Administrators group are enough.

Step 7: Choose the authentication method

Authentication methods compared

Set in POCKET KNIFE under Settings → Connection (or PK_WINRM_AUTH). All methods encrypt the content – also over HTTP 5985.

Method	How POCKET KNIFE signs in	Needs on the client	Assessment
Kerberos	ticket from the domain controller, the service account password never leaves the server	DNS names of the clients, synchronized time, Kerberos not disallowed by GPO	recommended
Negotiate	Kerberos where possible – otherwise NTLM automatically (e.g. access by IP address)	Negotiate not disallowed by GPO	good transition
NTLM	challenge/response with the service account	Negotiate not disallowed by GPO; NTLM not blocked domain-wide	deprecated
CredSSP	passes the credentials on to the client (delegation)	"Allow CredSSP authentication" enabled	only if needed

Authentication methods

In POCKET KNIFE under **Settings → Connection & accounts → Authentication method**. For new installations: **Negotiate**, and after a few days **Kerberos only** (see Kerberos instead of NTLM). Notes:

- **Kerberos** needs the computers by **DNS name** (comes from the AD sync), synchronized time and port 88 to the domain controllers. SPNs do not have to be created.
- If the service account is a member of **"Protected Users"** – highly recommended – **NTLM and CredSSP are blocked for the account**. POCKET KNIFE must then be set to "Kerberos only", the repository must be given with its DNS name and the account must support AES ("This account supports Kerberos AES 256 bit encryption"); otherwise sign-ins fail. Remote sessions via RDP with the service account then do not work (they use NTLM).
- POCKET KNIFE does **not** support **CredSSP** (any more): it would pass the service account's password to every client. Packages do not need it – POCKET KNIFE copies installation files to the client itself.

Optional: WinRM over HTTPS (5986)

Optional: WinRM over HTTPS (5986) – the path to the certificate

Group Policy does not create an HTTPS listener. Common approach: computer certificate by autoenrollment, listener by startup script.

1. **Certification authority** – template based on "Computer": purpose Server Authentication, subject = DNS name
2. **Autoenrollment** – GPO: Certificate Services Client – Auto-Enrollment enabled
3. **Create listener** – startup script: winrm quickconfig -transport:https -quiet
4. **Firewall** – rule for TCP 5986, remote IP: POCKET KNIFE server
5. **POCKET KNIFE** – PK_WINRM_HTTPS=true, PK_WINRM_CA_FILE = root certificate (PEM)

WinRM over HTTPS

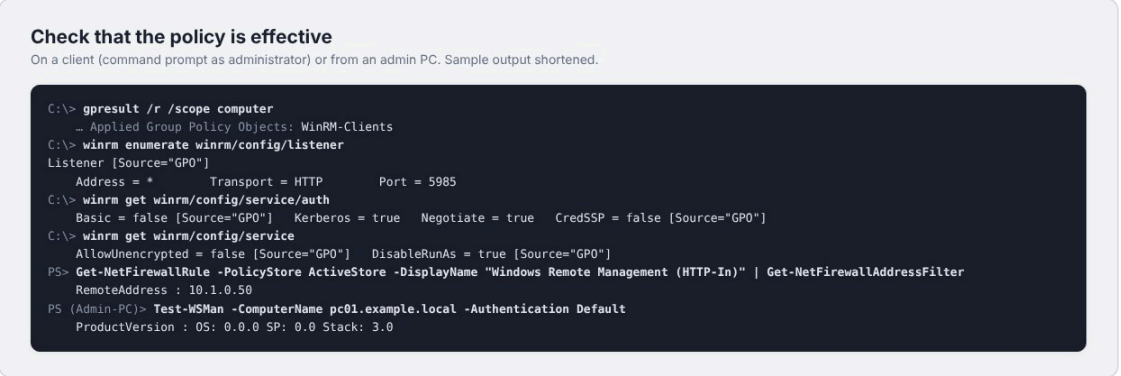
Over HTTP (5985) the content is already encrypted. HTTPS additionally protects the sign-in itself and proves the client's identity to the server by certificate – useful in networks with high protection requirements. Group Policy cannot create an HTTPS listener; the usual way:

1. In the certification authority, a certificate template based on "Computer" with purpose **Server Authentication**, subject = DNS name; Read and Enroll for "Domain Computers".
2. Group Policy: *Windows Settings → Security Settings → Public Key Policies → Certificate Services Client – Auto-Enrollment* → **Enabled**, tick "Renew expired certificates …" and "Update certificates …".
3. Listener by **startup script** (*Windows Settings → Scripts → Startup*), e.g. `winrm quickconfig -transport:https -quiet` – it uses the matching computer certificate. After a certificate renewal the listener must get the new certificate (run the script again or recreate the listener).
4. Firewall rule for **TCP 5986** as in step 4.
5. In POCKET KNIFE (`.env`): `PK_WINRM_HTTPS=true` and `PK_WINRM_CA_FILE=/data/winrm-ca.pem` (root certificate of the CA in PEM format) – POCKET KNIFE then verifies the clients' certificates.

Optional: logging

- *Windows Components → Windows PowerShell →* ***Turn on PowerShell Script Block Logging*** – the event log "Microsoft-Windows-PowerShell/Operational" (event 4104) then shows which commands ran via WinRM. That is valuable for traceability; keep in mind that POCKET KNIFE's scripts and those of your packages appear there too (do not write passwords into package scripts).
- "Microsoft-Windows-WinRM/Operational" shows connections and errors, the Security log (event 4624, logon type 3) the service account's sign-ins with the method (Kerberos/NTLM).
- These logs are suitable for central collection (Windows Event Forwarding or a SIEM).

Step 8: Check



Check

1. On a client: `gpupdate /force`, then `gpresult /r /scope computer` – the policy must be listed under “Applied Group Policy Objects”.
2. `winrm enumerate winrm/config/listener` – a listener with `Source="GPO"`; `winrm get winrm/config/service` – `AllowUnencrypted = false`, under `auth` `Basic = false`, each with `[Source="GPO"]`.
3. Firewall: `Get-NetFirewallRule -PolicyStore ActiveStore -DisplayName "Windows Remote Management (HTTP-In)" | Get-NetFirewallAddressFilter` shows the allowed remote address.
4. From **another** computer: `Test-NetConnection pc01 -Port 5985` must **fail** (the firewall works).
5. In POCKET KNIFE: **Settings → Maintenance → Test connection** with a computer name (WinRM, administrator rights, ADMIN$) and **Authentication method → Test**.

Common errors

Message / behavior	Cause and fix
Timeout, “WinRM connection failed”	firewall: rule missing, wrong profile (network not recognized as “Domain”) or wrong remote IP (enter the Docker host’s address)
“The WinRM client cannot process the request” / connection refused	no listener: the policy from step 3 is not applied or the IPv4 filter is empty
401 / access denied	the service account is not a local administrator, or the authentication method is blocked on the client (Kerberos/Negotiate “disallow” enabled, CredSSP not allowed)
Kerberos: “no SPN”, “server not found”	access by IP address or short name, DNS record missing or pointing to another computer
Worked, stopped after adding to “Protected Users”	NTLM/CredSSP are blocked for the account – switch to “Kerberos only”

Message / behavior	Cause and fix
“The service is configured to not accept any remote shell requests”	*Windows Remote Shell → Allow Remote Shell Access* was disabled – set it back to “Not configured”

System requirements and sizing

Docker host

- Linux (e.g. Debian 12/13 or Ubuntu 22.04/24.04), 64-bit (x86_64), VM or physical
- Docker Engine 24 or newer
- **SSD storage** for the data directory (`PK_DATA_PATH`) – the SQLite database writes many small changes during scans
- Domain DNS (`PK_DNS_1`), network access to the clients (WinRM 5985/5986, SMB 445, for remote sessions 3389/5900), the domain controllers (LDAPS 636) and the repository share
- Internet is optional: only for the winget index (`cdn.winget.microsoft.com`) – or upload the index offline; clients download winget packages themselves

Clients: Windows 10/11 or Windows Server 2016–2025 with PowerShell 5.1, WinRM enabled, service account as local administrator. winget from Windows 10 1809 / Server 2019.

Guideline values by environment size

> These values are estimates based on the program’s architecture (one process, SQLite, parallel WinRM connections), not measured in large environments. Start small, watch **Settings → Maintenance** and `docker stats`, and increase as needed.

Computers	vCPU	RAM (total, incl. guacd)	Storage (SSD)	Parallel scans / installations	Heartbeat
up to 250	2	2 GB	10 GB	6 / 8 (default)	5 min
250 – 1,000	2 – 4	4 GB	20 GB	10 / 12	5 min
1,000 – 3,000	4	8 GB	40 GB	15 – 20 / 15 – 25	10 min
3,000 – 5,000	8	16 GB	80 GB	20 – 30 / 25 – 40	15 min

Computers	vCPU	RAM (total, incl. guacd)	Storage (SSD)	Parallel scans / installations	Heartbeat
more than 5,000	–	–	–	several instances (e.g. per site/tenant)	–

Parallel scans and installations and the heartbeat interval are set under **Settings → Schedules & Performance** (changes to parallelism take effect after restarting the container).

What the resources are used for

Area	Requirement
Database	roughly 0.5 – 2 MB per computer (software, services, hotfixes, processes, hardware, tasks, history). 1,000 computers ≈ 1 – 2 GB. Shorter retention (log, scan log, deployments) keeps it small.
winget index	about 0.5 – 1 GB in the data directory, plus some RAM while it is being read
Scans	the work runs on the client; the server mostly waits for WinRM. A full scan takes about 20 – 60 s per computer – with 6 parallel scans roughly 400 – 1,000 computers per hour. More parallelism = faster, but more RAM and more load on DCs and network.
Heartbeat	checks with 48 parallel connections; a switched-off computer costs up to about 4.5 s. With many computers choose a longer interval so one run finishes before the next starts.
Deployments	repository files flow **through the Docker host** to the client (read SMB → write ADMIN$). 100 computers × 500 MB = 50 GB of network traffic via the host – for large packages plan 1 Gbit/s or more, limit parallelism or use winget (clients then download themselves).
Remote sessions (guacd)	about 50 – 150 MB RAM per concurrent RDP/VNC session and noticeable CPU when the picture moves; 10 concurrent sessions ≈ 1 – 1.5 GB and 1 – 2 vCPU extra

Tips for large environments

- Stagger scans with **scan profiles**: the full profile e.g. weekly, “Applications” daily; “Scan age” instead of a fixed time spreads the

load.

- Schedules with a **time window** and heartbeat instead of all computers at once.
- Adjust retention under **Settings → Schedules & Performance** (log, scan log, deployments).
- Optionally limit resources in Compose, e.g. `mem_limit: 8g` and `cpus: 4` for the `pocketknife` service.
- **Backup:** put the automatic backup (Settings → Backup) on another drive (`PK_BACKUP_PATH`) and plan about the database size (compressed usually much less) × number of versions.

Database cleanup: Old entries are deleted every hour – log (default 90 days), scan log (14 days), deployments including output (180 days), software changes (365 days), expired sessions and leftovers without a reference. The periods are set under **Settings → Schedules & Performance**. Once a day (default 3:00 AM) the **database maintenance** follows: update statistics, write back the WAL file and – if more than 20 % of the file is free – `VACUUM`, which shrinks the file. Size, free share and the last maintenance are shown under **Settings → Maintenance**; **Clean up now** starts the maintenance immediately. Deleted computers lose their inventory data at once. The log files under `/data/logs` rotate at 10 MB (at most 10 files).

Installation

Recommended folder structure on the Docker host – the data is kept outside the version folders:

```
/opt/pocketknife/
├── data/                  Database, logs, certificate, key
└── pocketknife-2.4/       current version with the .env
```

```
        cd /opt/pocketknife/pocketknife-2.4
        cp .env.example .env
        nano .env                  # domain, DCs, DNS servers, access group,
data path
        docker login ghcr.io -u <GitHub user>       # only for a private
registry (token with read:packages)
        docker compose up -d       # pulls
ghcr.io/soleil0815/pocketknife:latest (amd64/arm64)
```

The default is the ready-made image from the registry (`ghcr.io/soleil0815/pocketknife:latest`). With `:latest` a container manager such as Dockhand or Watchtower can update POCKET KNIFE **automatically** (or by hand `docker compose pull && docker compose up -d`); to stay on a version use `PK_IMAGE=ghcr.io/soleil0815/pocketknife:<version>` in the `.env`. Without internet: `docker build -t pocketknife:local .` and `PK_IMAGE=pocketknife:local`.

Then open `https://<docker-host>:8443` and sign in with an account from the access group. On first start, a self-signed certificate is generated in `data/tls/`; upload your own certificate under **Settings → Certificate (HTTPS)** (see Your own certificate).

> **Tip:** The service account does not need to be in the `.env`. Add it after your first sign-in under **Settings → Connection & Accounts** – it is verified before saving and stored encrypted. For the first sign-in without AD access, an optional emergency access is available (`PK_BREAKGLASS_USER` / `PK_BREAKGLASS_PASSWORD`). It is not subject to AD account lockout: choose a long random password (at least 16 characters) and enable the alert **"Emergency access used"**. Failed attempts are limited per address; each one is reported.

Update

The easiest way (from 2.24.1) is to update **always in the same folder**: put the ZIP into the program folder and run there:

```
cd /opt/pocketknife
sh update.sh pocketknife-2.24.1.zip
```

After confirmation (**y**/N), the script replaces the program files in the same folder – `.env`, database (`data/`), backups (`data-backup/`) and repository (`repo/`) stay untouched, no new version folder is created. The previous program files are kept in `.pk-vorher/` until the next update. It then builds, restarts, waits for a clean start and runs the self-test; if the build fails or the new version does not start, it returns (after asking) to the previous state; if it is interrupted in the middle of swapping (Ctrl+C, disk full), it puts the previous files back automatically. Running the same version again leaves `.pk-vorher/` unchanged. Finally it cleans up – asking each time – version folders of earlier updates (`pocketknife-2.23.3/`,

`pocketknife-2.24/`), the ZIP and unused Docker images; folders containing the database, backups or repository are never deleted – not even if files are still left in their `data/`, `data-backup/` or `repo/`. If POCKET KNIFE still runs from such a version folder, the script takes over its `.env` into the program folder. If the new script is not yet in the program folder: `unzip -oj pocketknife-2.24.1.zip pocketknife/update.sh -d /opt/pocketknife`.

With a ready-made image from the registry (from 2.27.1): instead of fetching the ZIP file and building the image on the Docker host, `update.sh` pulls the ready-made image from the GitHub Container Registry – for **amd64 and arm64**, Docker picks the right one:

```
cd /opt/pocketknife
sh update.sh --pull            # newest version
sh update.sh --pull 2.27.1     # a specific version
```

The script pulls the image, takes the program ZIP **from the image** (checked by SHA-256; image and ZIP versions must match), replaces the program files as with the ZIP way and starts **exactly this image** (by digest, without building) – with health check, self-test and the way back as above. Afterwards the container runs – like other containers – under the name from the registry, e.g. `ghcr.io/soleil0815/pocketknife:latest` (stored as `PK_IMAGE` in the `.env`; the ZIP way builds locally as `pocketknife:local` again). For the way back it keeps the previous image as `pocketknife:previous`; going back then needs neither a build nor the internet. If exactly this image is already running, it reports “already up to date”.

- **Registry:** default `ghcr.io/soleil0815/pocketknife`; your own (e.g. a fork or a mirror registry) in the `.env` with `PK_IMAGE_REPO=ghcr.io/<owner>/pocketknife`.
- **Private registry:** sign in once on the Docker host as the user running the update (usually root): `docker login ghcr.io -u <GitHub user>`, password = **personal access token** with `read:packages` (for a fine-grained token: access to the repository, “Packages: Read”). Docker remembers the sign-in; the token is not in the `.env`.
- **Without internet** or with your own code changes, the ZIP way stays as before (builds locally).
- The first switch to 2.27.1 is done with the ZIP file (the old `update.sh` does not know `--pull` yet); after that `--pull` is enough.

Where the image comes from: for every version GitHub (workflow `.github/workflows/release.yml`) builds the image for amd64 and arm64, runs the self-test in both, checks both with **Trivy** for known vulnerabilities and only then uploads it (additionally every Monday automatically with updated packages – "Wöchentliches Sicherheitsupdate"; with `:latest` the containers are thus up to date every week); plus a release with the ZIP file and its checksum. It is triggered by a tag `v<version>` that must match the file `VERSION`. Every push and pull request is additionally built and tested (`test.yml`) without publishing anything.

Manually it works as before – extract and run in the new folder with the old one as a parameter:

```
cd /opt/pocketknife
unzip pocketknife-2.24.1.zip && mv pocketknife pocketknife-2.24.1
cd pocketknife-2.24.1
./update.sh /opt/pocketknife
```

To return to the previous state manually: copy the files from `.pk-vorher/` back into the program folder and run `docker compose up -d` (built locally: first `docker build -t pocketknife:local .`).

Deployments still running during the update are interrupted; POCKET KNIFE then cleans up on the computers and waits for installers that are still running (see Restart during a deployment).

Pinned versions (reproducible build): base image with exact Python version and digest, Debian packages via a fixed point in time on snapshot.debian.org and all Python packages including dependencies are recorded in `Dockerfile`, `requirements.lock`, `build-requirements.txt` and `docker/apt-*.txt`. A rebuild therefore always yields the same image. To pin or deliberately update:

```
./tools/pin-versions.sh           # pin the versions of the built image
./tools/pin-versions.sh --update  # fetch the newest versions and pin them
```

The tool then rebuilds completely as a check and runs the self-test. If `requirements.lock` is empty, the image is built from the minimum versions in `requirements.txt` with a notice.

First Steps

1. **Settings → Connection & Accounts**: enter the service account and click “Check & save”, then “Test LDAPS”.
2. **Settings → Active Directory**: include or exclude containers and click “Sync now”.
3. **Settings → Maintenance → Test connection** with a computer name (WinRM, administrator rights, ADMIN$).
4. **Settings → Repository & winget**: enter the repository path and click “Load index now”.
5. Computers are now scanned automatically as soon as they are online (profile “Default”).

Using the Interface

Navigation and Language

The left side lists the sections Overview, Computers, Software, Packages, Deployments, Schedules, Scan profiles, Tools, Kiosk, Reports, Log, Settings and Help. At the bottom left, **DE / EN** switches the interface language and ☀ / ☾ switches between light and dark mode; both apply to this browser. "As system" can be restored under **Settings → Maintenance → Appearance**.

To change the width of the collection tree (and the report list), drag its right edge; double-click the edge = default width.

Almost every page has a **?** at the top right that jumps directly to the matching chapter of this help.

News and updates

After an update, a discreet **New** appears below the version number at the top left – with security updates **New · Security** with a red dot. One click opens the news: short points per version, marked as **Security**, **New**, **Improved** or **Fixed** (security always at the top). Afterwards the hint disappears; the news remains available via **News** in the same place. POCKET KNIFE remembers per user what they have seen – anyone who missed several updates sees all of them since. All details remain in the CHANGELOG.

New version available (main admins only): the POCKET KNIFE server checks GitHub for the newest release once a day – never the browser, and through the proxy server if one is set. If there is a newer version, it shows **↑ 2.28.1 available** (red if it contains security updates). The click shows its news and the command for the update on the Docker host (`sh update.sh --pull`, see Update); it is logged once.

Settings under **Settings → Maintenance → News and updates**:

Field	Meaning
Check	ask once a day for new versions (can be switched off)

Field	Meaning
Repository	GitHub repository with the releases, e.g. `soleil0815/pocketknife` (or your own fork)
Read token	only for a private repository: fine-grained token with read access to the repository (“Contents: Read”). Stored encrypted, never shown, never in the diagnostics package
Check now	ask immediately; the status shows the result or the reason (token invalid, repository not found, no connection)

Without internet access or without a token nothing happens – no error in the interface.

Tables

All lists work the same way:

- **Sort:** click the column header.
- **Move columns:** drag the column header with the mouse – onto the left half of a column = before it, onto the right half = after it. Or in the **Columns** button drag the row by ⠿ or use ▲▼ (left/right) – handy on small screens when the table scrolls sideways.
- **Show/hide columns: Columns** button – the window stays open for several changes; “Show all”, “Reset” (order, visibility and widths).
- **Column width:** drag the right edge of the column header (content that is too long is shortened with …); **double-click** the edge = automatic width again.
- **Filter:** ▾ in the column header – by values (checkboxes) or text; “(Empty)” and “(Not empty)” are also available. Active filters are indicated by the “Reset column filters” button.
- **Search:** the “Filter …” field above the table searches all columns.
- **Selection:** click selects, **Ctrl**+click adds, **Shift**+click selects a range, **Ctrl+A** selects all. **Double-click** opens.
- **Right-click** shows all actions for the selection.

The browser remembers column order, widths, hidden columns and sorting for each view.

Keyboard Shortcuts

Key	Action
Ctrl+A	select all
Del	delete selection (with confirmation)
Ctrl+C	copy names of the selection (computer list)
Ctrl+H	heartbeat for the selection (computer list)
Ctrl+X / Ctrl+C / Ctrl+V	cut / copy / paste collection (collection tree)
F2	rename collection
Esc	close dialog or menu

Computers and Collections

Computer List

The **collection tree** is on the left, the computers of the selected collection on the right. A green dot indicates that a computer is online. **Right-click** one or more computers to access: Details, Scan ▸ profile, Heartbeat, Deploy, Tools ▸, Remote Desktop, Add to collection ▸, Remove from collection, Copy name, CSV, “Scanning allowed” and Delete.

Clearing “Scanning allowed” blocks automatic scans for a computer (e.g. for sensitive servers); manual scans remain possible.

Yellow dot – address not confirmed: POCKET KNIFE only shows a computer as online if this very computer answers at its address. For a new address (and at the latest after 24 hours) it asks the computer via WinRM for its name and domain. If another device answers (e.g. because DHCP has meanwhile given a switched-off PC’s address to a printer or phone), a device outside the domain, or – at a previously different address – a device without WinRM, the computer counts as offline. The dot turns yellow, the computer view names the reason (e.g. “At 192.168.0.77 ‘PRINTER-3F’ answers instead of PC-0004”) and offers “Check again now”. Scans and deployments do not run then – they would hit the wrong device. The built-in collection **“Wrong address”** lists all affected computers; the column “DNS address” (can be shown) displays the currently resolved address.

The cause is almost always an outdated record in the domain’s DNS. Remedy: switch the computer on (it registers again), correct the record and permanently enable **DNS scavenging** on the domain controllers. Settings: **Settings → Name resolution**:

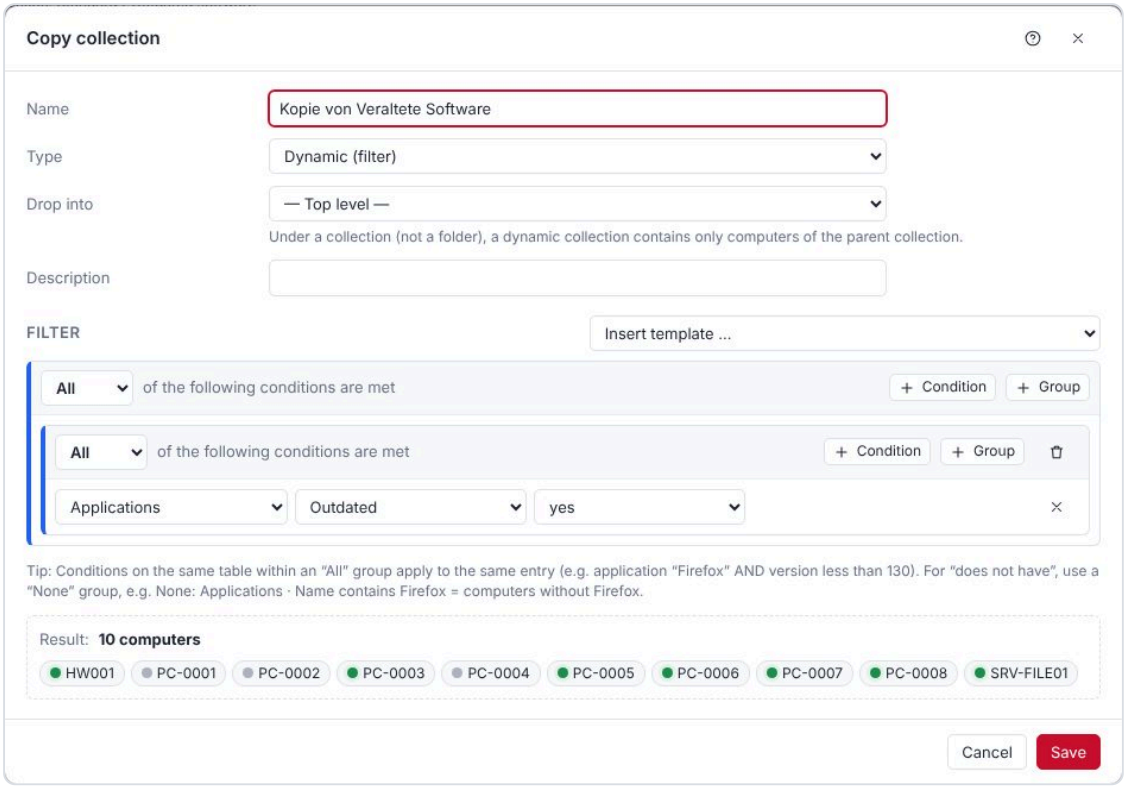
- **DNS cache (min.):** how long a resolved address is valid in POCKET KNIFE; **“Flush DNS cache”** discards all immediately (0 = resolve anew at every heartbeat).
- **Query DNS directly:** ask the domain’s DNS servers (default: the domain controllers) directly – bypassing caches on the Docker host. “Test resolution” shows the result and warns if the system returns a different address.

- **Check identity** (on by default) and **Confirm again after (hrs.)**. Devices with a problem are asked again at most hourly; “Check again now” or a manual heartbeat checks immediately.

Collections

There are three types:

- **Folders** structure the tree.
- **Static collections** contain computers selected by hand. You can **drag** computers from the list onto a static collection.
- **Dynamic collections** populate themselves through filters on all scan data.



Filter of a dynamic collection

Filters consist of nested groups: **All** (AND), **Any** (OR), **Not all**, **None**. Conditions on the same table within an *All* group refer to the **same** entry.

Example – “7-Zip missing or outdated”

```
Any
├── None:   Applications · Name · starts with · 7-Zip
└── All:    Applications · Name · starts with · 7-Zip
            Applications · Version · Version less than · 24.08
```

Example – “Windows 10 with low disk space”

```
All
├── Computer · Operating system · contains · Windows 10
└── Disks · Disk · is · C:   and   Disks · Free (GB) · less than · 15
```

A dynamic collection *under* another collection (not under a folder) contains only computers that are also in the parent collection.

Shortcut – straight from the inventory: Right-click an entry (e.g. a program, printer, service or hotfix in the computer view, a program in the software overview, a cell in the computer list, or a value on the *General* page such as model or operating system) → **Create dynamic collection**. The options include:

- for programs at the top **“name” installed** and **“name” not installed** – the collection gets a matching name right away, e.g. “PdfCraft installiert” / “PdfCraft nicht installiert” in German or “PdfCraft installed” / “PdfCraft not installed” in English (no prefix like “Applications:”). If the name contains a version number, this applies to any version (`7-Zip*`); “Computers with exactly …” uses the full name,
- *Computers with “name”* – for programs also **any version** (the name without the version number as a pattern, e.g. `7-Zip*`), one specific version or **older than** that version,
- the clicked column (e.g. a printer’s driver) on its own or together with the name,
- *Computers without …* (group *None*),
- with several entries selected: with at least one, with all, or without these entries.

The normal collection dialog then opens with the finished filter – adjust name, location and conditions before saving.

Usage: Right-clicking in the tree offers Open, Edit, New ▸, Duplicate, Scan collection, Heartbeat, Deploy, Tools, Report, Cut/Copy/Paste, Rename and Delete. Dragging moves a collection; holding **Ctrl** copies it. Default and AD collections are read-only and fixed in place – they can only be expanded and collapsed, not dragged; “Duplicate” or Copy/Paste creates an editable copy.

Delete protection: a collection that is still used by another collection (filter “Member of”), a schedule, scan profile, tenant, custom report, report mailing or notification cannot be deleted – the message names where it is used. The same applies to custom fields. Circular references (A refers to B, B to A) are rejected when saving.

Collections that build on each other: Up to 12 references in a row are possible (collection A is “member of” B, B of C … – child collections count too); within one collection up to 10 levels of filter groups are possible – both are counted separately. When saving or moving, POCKET KNIFE also checks all collections that build on the changed one: if one of them would stop working, nothing is saved and the message names the affected collection. This way no scan profile or schedule silently loses its targets.

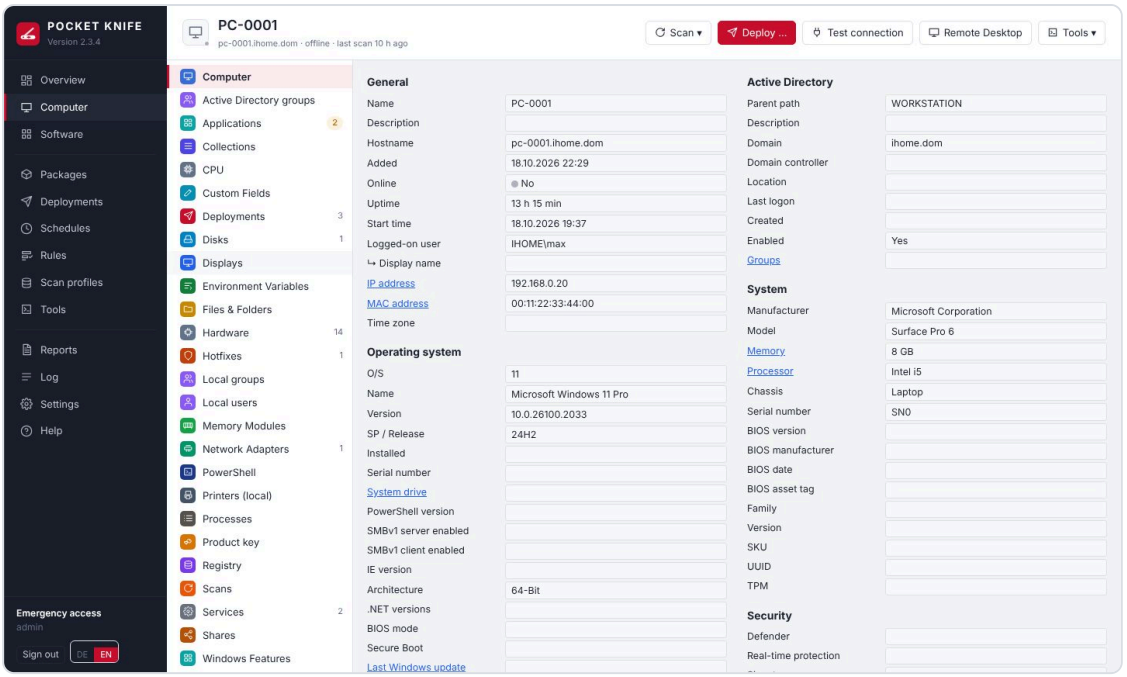
Regular expressions in filters: at most 200 characters; nested repetitions such as `(a+)+` or `(a|b)*` and backreferences are not allowed because they can slow evaluation down dramatically.

AD Collections

If enabled under **Settings → Active Directory**, the sync creates the OU structure under an entry named after the domain, plus a “Groups” folder with all AD groups that contain computers. POCKET KNIFE maintains these collections itself; in the tree they can only be expanded and collapsed, not moved.

Computer View

Double-clicking a computer opens its detail view. The categories are on the left (Computer, Applications, Hardware, Services, Disks, Network Adapters, Scans, Deployments …), the data on the right. The overview shows the operating system, hardware, security (Defender, BitLocker, TPM, firewall) and scan status.



Detail view of a computer

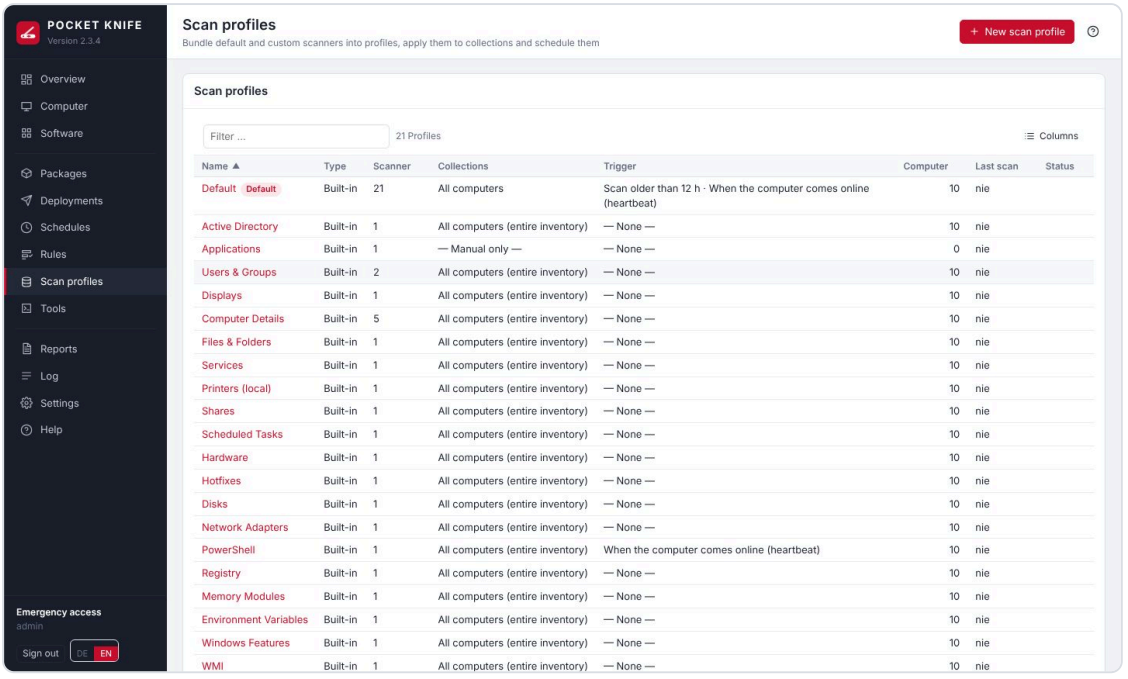
Actions directly on the computer:

Where	Action
Applications	Update (winget), Uninstall now, Create uninstall package (right-click)
Services	Start, Stop, Restart, Change startup type
Processes	Load live and end
Overview → Passwords & Keys	Show LAPS password and BitLocker recovery key (every retrieval is logged)
Header	Scan, Deploy, Tools, Remote Desktop

Scan Profiles and Scanners

Scan Profiles

A scan profile defines **what** is scanned (scanners), **on which computers** (targets: collections and/or individual computers) and **when** (triggers). Built-in profiles are included for each scanner (Active Directory, Applications, Computer Details, Disks, Displays, Environment Variables, Files & Folders, Hardware, Hotfixes, Memory Modules, Network Adapters, PowerShell, Printers, Registry, Services, Shares, Users & Groups, Windows Features, Scheduled Tasks, WMI, Disk space), as well as **Default**.



Scan profiles

- Built-in profiles cannot be deleted and their scanners cannot be changed; **triggers and targets** can be configured freely. For custom combinations, use **Duplicate**.
- **Targets** (tab in the profile): **All computers**, any collections (including dynamic ones) and/or **individual computers**. The collections appear grouped by folder and in alphabetical order, like the tree on the left (Default, AD, your own folders); **Search collection** filters across all folders, **selected only** shows the selected ones, and they are listed at the top for removal. Collapsed folders show how many are selected

inside. The same applies to the targets of schedules. Target computers are the members of the selected collections plus the individually selected ones. New profiles initially target “All computers”. Without a target, a profile only runs manually (Scan → choose profile). The **Computers** tab shows which computers the profile currently covers.

- **Default** contains all scanners except Files & Folders, PowerShell, Registry, WMI and Disk space. New computers are scanned with it the first time they come online.
- **Disk space** (optional scanner): determines the folder sizes of the fixed drives as a chart – like the “Analyse disk space” tool, but schedulable. Deliberately **not** in the standard scan because it puts more load on the client; schedule it specifically with its own trigger (e.g. monthly, at night) and collections. The depth is set under **Settings → Schedules & performance**; results appear on the computer under **Disk space** and in the chart.
- After every installation or uninstallation, the **Applications** profile runs automatically (can be changed in the package).

Triggers

Trigger	Meaning
Once	at a fixed point in time
Interval	every n minutes/hours/days
Daily / Weekly / Monthly	at a time of day on specific days
Scan age	as soon as the last scan is older than a given period, optionally only within a time window
Heartbeat	as soon as a computer is (back) online

“Catch up on offline computers” scans computers that were offline at the scheduled time as soon as they come online.

Custom Scanners

Under **Scan profiles → Custom scanners** you create scanners of type Registry, Files & Folders, WMI or PowerShell. Each custom scanner runs in the built-in profile of its type (e.g. “PowerShell”) and additionally in your own profiles. The result appears on the computer in the category of the same name. If there are several scanners, pick one at the top from the

drop-down list (or ‹ ›) or show them all with “All scanners”; the selection is remembered per category. **Test** runs a scanner immediately on a computer and shows the result.

> **Important:** The built-in “PowerShell” profile initially has no trigger. To run custom PowerShell scanners regularly, set a trigger there (e.g. daily) – or add the scanner to a custom profile that has a trigger.

Examples

Type	Input	Result
Registry	`HKLM:\SOFTWARE\Contoso\Agent`	all values of the key
Files & Folders	`C:\Program Files\Contoso\*.exe`	size, date, file version
Files & Folders	`%SYSTEMDRIVE%\Users\*\ntuser.dat`	one entry per user profile
WMI	`SELECT Name, EstimatedChargeRemaining FROM Win32_Battery`	battery level
PowerShell	see below	one row per output object

```
        # Version and path of a company application
        Get-ItemProperty HKLM:\SOFTWARE\Contoso\App | Select-Object Version,
InstallPath

        # Last boot as text
        "Last boot: " + (Get-CimInstance
Win32_OperatingSystem).LastBootUpTime

        # Local administrators
        Get-LocalGroupMember -SID S-1-5-32-544 | Select-Object Name,
PrincipalSource
```

Objects become columns, text becomes the “value” column. Write-Host, hashtables and Format-Table are captured; errors appear in the “error” column.

Paths for “Files & Folders”: Environment variables are resolved on the client – `%SYSTEMDRIVE%`, `%ProgramData%`, `%ProgramFiles%`, `%windir%` or in PowerShell notation `$env:SystemDrive`. The scan runs as the service account or SYSTEM, so `%USERPROFILE%` and `%APPDATA%` point to that account’s profile, not the logged-on user’s – for all users use `%SYSTEMDRIVE%\Users\*\…`. Wildcards: `*` and `?` for a name on one level, `**` for any number of folder levels (e.g. `%ProgramData%\Contoso\**\*.log`). `**` searches everything below

and is much slower for large folders like `C:\Users` – where the depth is known, use `*` instead. “Search subfolders recursively” additionally returns all content below.

Sizes: POCKET KNIFE shows byte values in readable form as KB, MB, GB or TB (tooltip: exact byte value). Setting per scanner under **Sizes**:

Setting	Effect
detect automatically (default)	columns whose name suggests a size (e.g. `Size`, `Length`, `FreeSpace`, `Capacity`) if all values are plain numbers and at least one is ≥ 1024. Columns like `SizeGB`, `Count` or `Percent` stay unchanged.
only these columns	exactly the listed columns (comma-separated, e.g. `Size, FreeSpace`)
never convert	values as delivered

So the script should output bytes rather than already rounded GB values – then sorting, CSV export and filters in dynamic collections are correct. This also applies to the `size` column of the “Files & Folders” scanner.

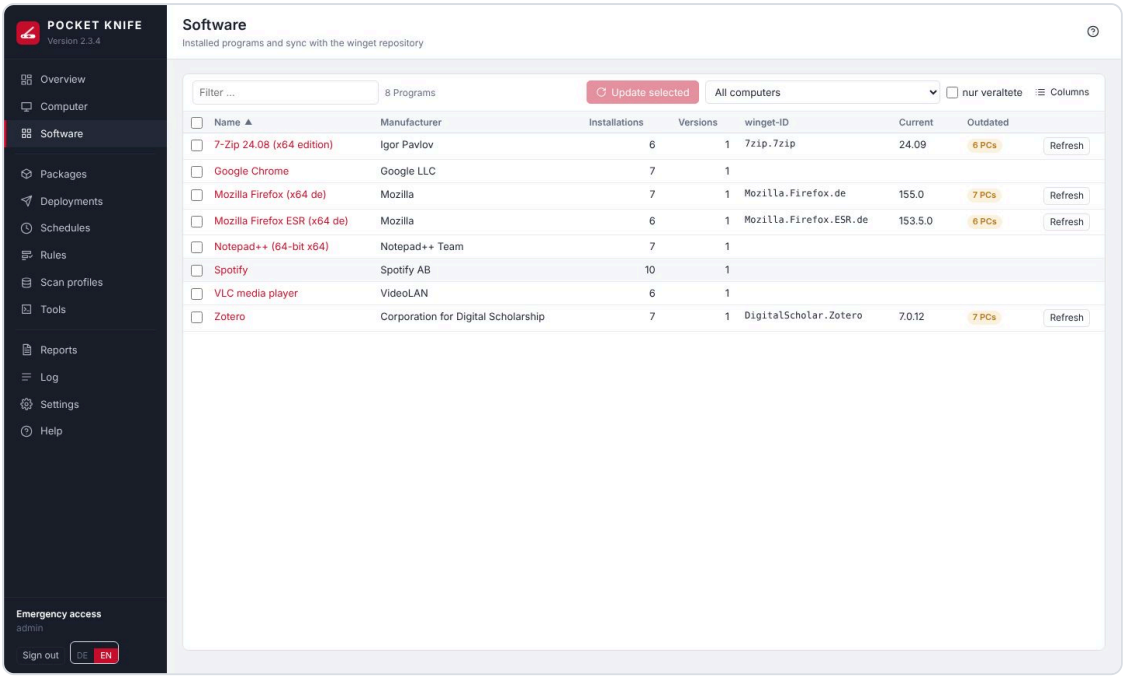
Dynamic collection from a size: right-click a value like “28 GB” → *Create dynamic collection* offers “Size ≥ 27.8 GB” and “Size ≤ 27.9 GB” (instead of the exact byte value); the value can be adjusted in the dialog, e.g. to `20 GB`. In general, numeric comparisons in collections understand units: `500 MB`, `20 GB`, `1.5 TB` (1 GB = 1024³ bytes).

Sizes in filters and reports: In the filter editor (collections and custom reports), size columns offer ≥, ≤, >, <, =, ≠ and accept values with a unit (`10 GB`); suggestions are readable thresholds. In custom reports sizes appear as KB/MB/GB and are sorted by size; the CSV export contains the byte value.

```
        # Size of user profiles in bytes – displayed automatically as GB/MB
        Get-CimInstance Win32_UserProfile -Filter "Special=False" | ForEach-
Object {
          [pscustomobject]@{
            Path     = $_.LocalPath
            Size     = (Get-ChildItem $_.LocalPath -Recurse -Force -File -
ErrorAction SilentlyContinue | Measure-Object Length -Sum).Sum
            LastUsed = $_.LastUseTime
          }
        }
```

Software

The software overview combines all installed programs on all computers and shows which version is current in the **winget catalog**. “Outdated” counts the computers with an older version.



Software overview

- **Update selected:** select programs (Ctrl/Shift) → button or right-click. For each winget ID, a `winget upgrade` deployment is created for all outdated installations.
- **Detail view (double-click):** all computers with their version; update, deploy or uninstall the selection; set the winget mapping manually or ignore the program.
- **Create uninstall package:** right-click a program. The package removes the program on any computers, collections or on a schedule (see the “Uninstall” step).

Update-check exclusions (your own list): programs that should never count as “outdated” – e.g. runtime libraries, driver tools or line-of-business applications whose version the vendor prescribes. Found under **Software → Exclusions …** and **Settings → Repository & winget**; faster via right-click on one or more programs → *Exclude from update check*.

Field	Example pattern	Effect
Name	`Microsoft Visual C++*`	all Visual C++ runtime packages
Name	`*Runtime*`	everything with “Runtime” in its name
Publisher	`Datev*`	all programs of this publisher
winget ID	`Oracle.JavaRuntimeEnvironment`	exactly this winget mapping

`*` stands for any number of characters, `?` for exactly one; case does not matter. When adding, a preview shows which installed programs are affected. Excluded programs are shown as **excluded** in the list, do not count as outdated (overview, collection “Outdated software”) and are not updated by winget schedules. The winget mapping stays visible; targeted deployment still works. Remove via the trash icon in the list – the programs are then checked again.

The older **Ignore** in a program’s details only removes its winget mapping; these programs are listed in the exclusion list under “Individually ignored programs”. - **Create package from winget:** right-click, if a winget ID is mapped.

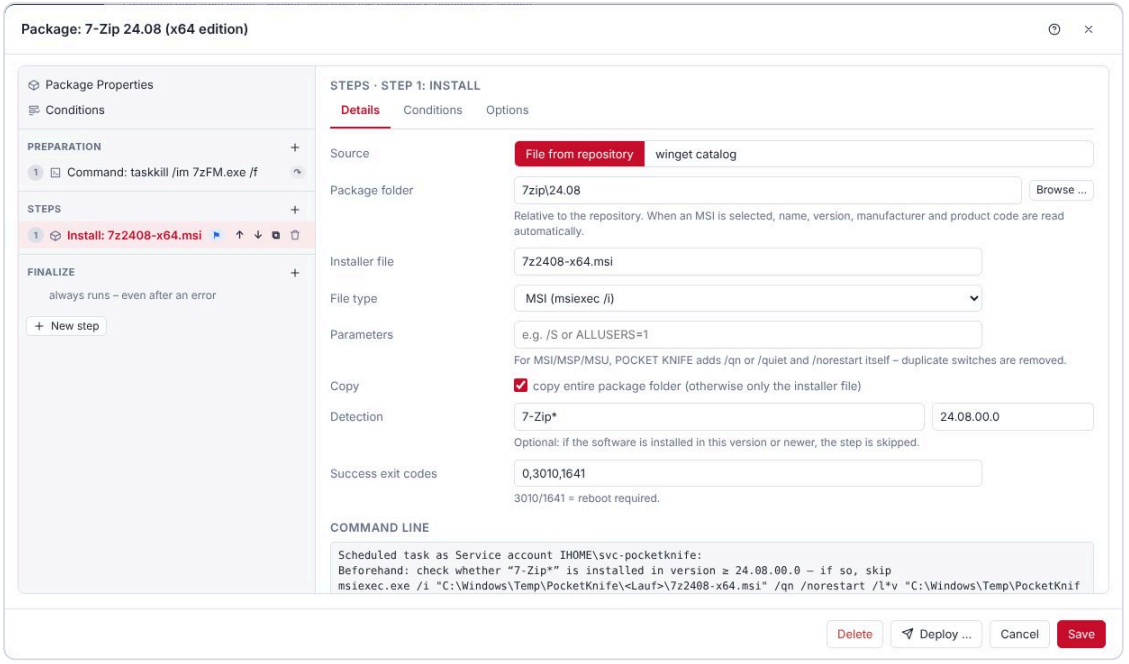
The winget mapping takes variants such as ESR, beta or language versions into account. Uncertain mappings are not updated automatically.

Packages

A package consists of **steps** in three sections:

Section	Purpose
Preparation	e.g. close program, remove old version
Steps	the actual work
Finalize	always runs – even after an error (cleanup, scan)

Each step can be named in the **Label** field at the top (e.g. “Install Wazuh agent”). The label appears in the step list and in the deployment log; if empty, the automatic description is used. A new package starts at the **package properties** with the name.



Package editor

New package: Packages → **New package**. The structure is on the left, the settings of the selected entry on the right. You can reorder, duplicate, disable and move steps between sections by dragging, with ↑/↓ or by right-clicking.

“Notification” column: A green ✓ in the package list shows that the signed-in user gets a Windows notification after this package – by the same rule as on the client: “always” in the package, or “as in the

settings", notifications are switched on there and the package really installs or updates a program (no uninstall, no plain script). The tooltip names the reason.

Search the repository: With "File from repository", **Browse ...** opens the repository. The search field at the top finds files and folders in the whole repository including subfolders (OU admins: in their tenant folder) – part of the name is enough (`7zip`), with `*` and `?` as patterns (`7z*.msi`). Clicking a file takes over folder and file. The search uses an index of the repository in the **server's memory** (not in the database; 10,000 files ≈ 2–5 MB): matches appear instantly, the index is refreshed in the background after 5 minutes and its age is shown below the search field ("As of: 3 min. ago"). **Re-read** fetches the current state immediately (main admins only). Repositories with more than 200,000 entries are not cached – every search then runs directly on the share (at most 15 seconds).

Export and import: Export saves the selected packages (without a selection: all shown; also via right-click) as a `*.pkpackages.json` file – e.g. for a second POCKET KNIFE installation or as a backup of single packages. Nested packages are included. The file contains no IDs: collections and scan profiles are matched by name. After choosing the file, **Import ...** first shows what will happen (new, as copy, skipped, error) – only **Import** creates the packages. The same checks apply as when saving in the editor; for a tenant, files must be in its repository folder. If a collection used in a condition does not exist, the package is rejected (simply dropping the condition would run it on more computers). Packages contain no passwords; scripts and commands are in the file exactly as in the package.

Break detection (repository files): When a package is saved, POCKET KNIFE remembers size, modification date and fingerprint (SHA-256) of every repository file the package uses (with "copy whole folder" also the folder's file list). Packages from before 2.26 get this state at their first check. Checks run every 5 minutes (from the repository index – name, size and date only), before **every** deployment, schedule run, kiosk installation and retry (directly on the share), and with **Check now** in the package. If only the date changed, the fingerprint decides – identical content does not count as a change.

State	Meaning	Consequence
✓	files unchanged	–

State	Meaning	Consequence
! Deviation	other files in the package folder changed or were added	run continues, note in the log
✕ Break	file or folder missing, or the installation file or the source of a file copy changed	deployments and schedules are **interrupted** (no computer is touched), warning in the log
?	repository not reachable	no break – the run reports the error itself

The **State** column in the package list shows the result (tooltip with the files), the **Overview** a tile “Packages with a break”. In the package, **Repository files** lists every deviation with previous and current state. If the change was intended (e.g. a new version copied over), **Accept new state** confirms the current content – for an MSI, POCKET KNIFE points out when the version in the package no longer matches the file. Missing files are not fixed this way: if they were only moved, **Find file …** finds them by fingerprint (repair), otherwise adjust the package. OU admins see details only for files in their tenant folders; a central package with a break is “currently not available” for them and only a main admin can accept it. Kiosk users only see “currently not available”.

Full check: Once a week every used file is additionally read completely and its fingerprint compared – this also finds a replacement with the same size and the same date. It runs at night in the time window (default 1 to 6 am), throttled (default 50 MB/s) and one file after the other; if the window ends, it continues the next night. Configurable under **Settings → Repository & winget → Package break detection**. Load: roughly like copying the used package folders once (network and disks of the file server); the clients notice nothing.

Moving the repository: Packages only store the path **inside** the repository (e.g. `7zip\24.08`), never the server. If the repository moves to another server, just change the path under **Settings → Repository & winget** – all packages keep working, imported ones too. Only paths you wrote into a command or script yourself (`\\server\share\…`) stay as they are; the export points them out.

Step Types

Step	What happens
Install	File from the repository (MSI, MSP, MSU, EXE, CMD/BAT, PS1, REG) or winget. A `.reg` file is applied with `reg.exe import` (like `regedit /s`, but with an exit code). For MSI files, name, version, manufacturer, product code and platform are read automatically when you select the file.
Uninstall	searches “Programs and Features” live for the display name (wildcard `*`) and uninstalls silently
Command	multi-line, like a .cmd file
PowerShell	any script; `exit 1` reports an error
Nested package	the steps of another package at this position
File copy	file/folder from the repository to e.g. `%ProgramData%\Contoso`
Registry	set values (REG_SZ, REG_EXPAND_SZ, REG_DWORD, REG_QWORD, REG_MULTI_SZ, REG_BINARY), delete values or keys, create keys – directly in the package, no file needed. **Import from .reg** turns the content of an exported .reg file into entries. 64- or 32-bit view (WOW6432Node) selectable. HKCU entries apply to the executing account – for user settings use *Run as: Logged-on user*.
Scan	inventory scan with a scan profile
Restart	restarts, waits until the computer is back and continues
Wait / Message / Sign out	pause, message to all sessions, sign out users

Computer name & co. in packages: The tool variables `$(Computer)` etc. only apply to tools. In package steps (command, PowerShell, paths) use the client’s Windows environment variables: `%COMPUTERNAME%`, `%USERDOMAIN%`, `%ProgramData%` in commands or `$env:COMPUTERNAME` in PowerShell. Note: `%USERNAME%` / `$env:USERNAME` is the **executing** account (service account or `SYSTEM`, computer account `PC-0815$`), not the logged-on user – unless *Run as: Logged-on user* is used. In PowerShell, `(Get-CimInstance Win32_ComputerSystem).UserName` returns the logged-on user.

Conditions

Conditions exist for the whole package and for each step. By default, **everything is selected**; clear only what does not apply:

- **Operating system:** Windows 10, Windows 11, Server 2016–2025
- **Architecture:** 32-bit, 64-bit, ARM64
- **Minimum PowerShell version**, **sign-in status** (always / only when a user is signed in / only when no user is signed in)
- **File** exists/missing/version, **Registry** exists/missing/value/version, **Collection** member/not member

If a package condition is not met, the computer is **skipped**; for a step condition, only that step is skipped.

Run as

Option	When
Service account (default)	almost always – full administrator rights; runs directly as a process of the service account, without a scheduled task and without a password on the client
Local System	for installers that require it; winget steps run this way automatically
Logged-on user	for programs in the user profile (e.g. per-user installations)

No password on the client (from 2.14): POCKET KNIFE starts steps as the service account via WinRM as a separate process (WMI `Win32_Process`). It runs with the rights of the WinRM logon, survives a dropped connection and is terminated together with its child processes on cancel or timeout. “Local System” and “Logged-on user” use a temporary scheduled task – also without a stored password. Tasks from older versions with a stored password are removed by the next scan. Because no credentials are stored on the client, a step running as the service account cannot reach network shares; POCKET KNIFE copies repository files beforehand itself. If a custom command still needs a share: run it as “Local System” and give the computer account (e.g. the group “Domain Computers”) read access.

PowerShell is always started by POCKET KNIFE with the full path and fixed switches:

```
%SystemRoot%\System32\WindowsPowerShell\v1.0\powershell.exe -NoLogo
```

`-NonInteractive -NoProfile -ExecutionPolicy Bypass` – for PowerShell steps, .ps1 installers and tools, whether as service account, Local System or user. Your own scripts do not need to handle this (no execution policy, no profile, no prompt). The setting applies to the package and can be overridden per step. **Running** shows the exact command executed on the client for each step, e.g.:

```
Directly as service account CONTOSO\svc-pocketknife (separate process, no
scheduled task):
msiexec.exe /i "C:\Windows\Temp\PocketKnife\<run>\7z2408-x64.msi" /qn
/norestart ALLUSERS=1 /l*v "...\pk_msi.log"
```

Examples

7-Zip from the repository (MSI)

1. New package → Install → Source “File from repository” → Browse → `7zip\24.08\7z2408-x64.msi`.
2. Name, version, detection and architecture (without x86) are taken from the MSI.
3. Save → Deploy.

Firefox via winget, closing the old version first

1. Preparation → Command: `taskkill /im firefox.exe /f`, Options → On error “Continue on error”.
2. Steps → Install → Source “winget catalog” → `Mozilla.Firefox` (empty version = always the latest).

Uninstall Zotero (including per-user installations)

Right-click “Zotero” in Software → “Create uninstall package …”. The step finds Zotero on every computer and calls `helper.exe /S` – for per-user installations, as the respective user.

Distribute a company configuration

1. File copy: source `Config\Agent`, destination `%ProgramData%\Contoso\Agent`.
2. Command: `reg add "HKLM\SOFTWARE\Contoso" /v Konfiguriert /d 1 /f`.
3. Finalize → Scan with the “Registry” profile.

Archive cannot be extracted (exit code -1978335140): Some winget packages (e.g. TeamViewer, Microsoft GameInput) come as a ZIP archive. As Local System, extraction via the Windows shell often fails. POCKET KNIFE then repeats the step once with `tar.exe` and sets `installBehavior.archiveExtractionMethod = tar` in the winget settings of the Local System account (`…\systemprofile\AppData\Local\Microsoft\WinGet\Settings\defaultState\settings.json`); existing settings are kept. Own settings files that cannot be read stay unchanged.

Proxy server

For networks without direct internet access: **Settings → Repository & winget → Proxy server**.

Field	Meaning
Proxy address	e.g. `http://proxy.firma.local:8080` – always with `http://`, also for HTTPS targets (the proxy sets up the encrypted connection via CONNECT). Empty = no proxy
User / password	optional, only for POCKET KNIFE itself (Basic authentication at the proxy). The password is stored encrypted in the vault and is never in the diagnostics package
Clients	optional: winget on the clients also downloads via the proxy

POCKET KNIFE itself downloads the winget catalog through the proxy. Without an entry the container's network configuration applies as before (environment variable `HTTPS_PROXY` in the container, if set). **Save & test** downloads the first bytes of the catalog and shows whether it works; "407" means the proxy requires authentication.

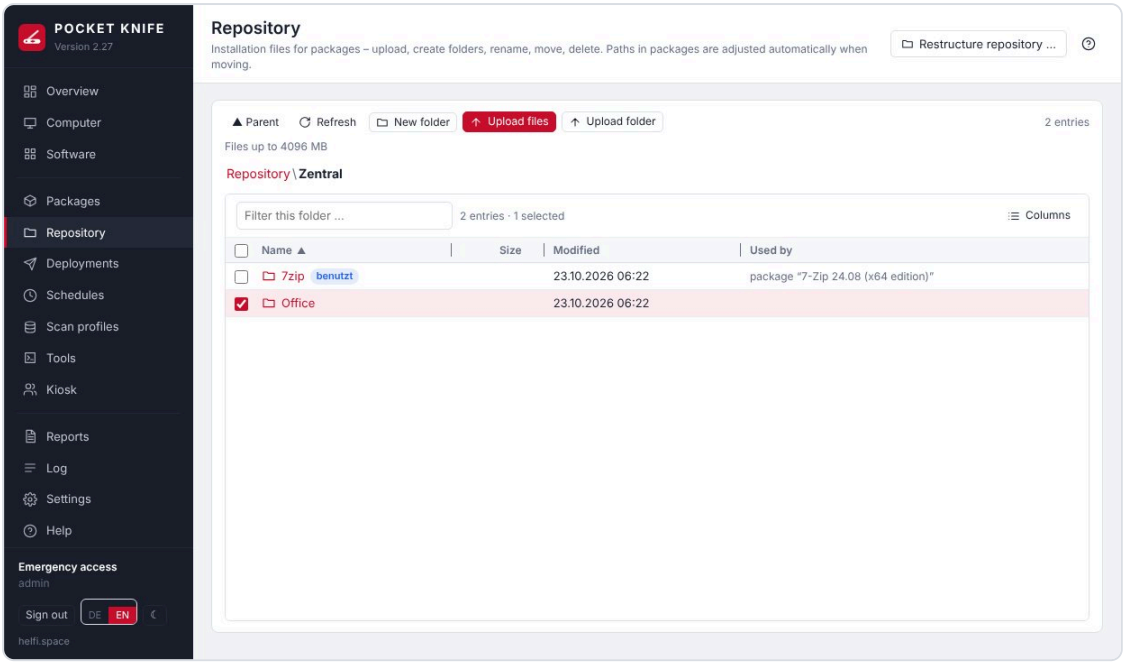
Clients: With the check box, POCKET KNIFE sets the winget setting `DefaultProxy` on the client with the next winget deployment (once, afterwards only on a change) and also uses the proxy when installing winget. Clearing the check box removes the setting again – but only if POCKET KNIFE set it itself (marker in the registry: `HKLM\SOFTWARE\PocketKnife`, value `WingetProxy`). **User and password never go to the clients.** If the proxy requires authentication, the clients sign in with their computer account – the proxy must allow Windows authentication (Kerberos/NTLM) for this or let the clients through by IP range without authentication.

winget on Servers

Before every winget step, POCKET KNIFE checks whether winget is present. If it is missing (Windows Server, older Windows 10 builds), the App Installer and its dependencies are installed – from a repository folder (offline) or directly from GitHub. winget is not available before Windows 10 1809 or Server 2019; winget steps are **skipped** there. You configure this under **Settings → Repository & winget**.

Repository

Under **Repository** (left menu) you manage the installation files directly in POCKET KNIFE – nobody needs access to the share for this. Writes always use the repository account (service account) on the server, never the operator's rights. **Prerequisite:** the account has **read and write** access to the share.



Repository view

The view works like the file explorer for clients:

- **Open:** double-click or click the folder name; the path at the top jumps back.
- **Select:** click, Ctrl+click (add), Shift+click (range), Ctrl+A.
- **Upload: Upload files**, **Upload folder** (with all subfolders) – or simply drag files and folders from the desktop into the list or onto a folder. Large files are sent in 8 MB pieces; if the connection drops, **Resume** appears in the list (also after reloading the page: choose the same file again, it continues where it stopped). POCKET KNIFE cleans up aborted uploads after 24 hours. Data travels from the browser through the POCKET KNIFE server (Docker host) to the file server – large files load both network paths.
- **Move:** drag entries onto a folder or onto a part of the path at the top, **Cut** and **Paste** in the target folder, or right-click → **Move to ...**.

- **Right-click:** Move to, Cut, Rename, New folder, Who uses this?, Delete, Copy path.

Paths in packages follow. When moving or renaming, POCKET KNIFE adjusts the paths in all affected packages (including disabled steps), as well as tenant folders, scheduled deployments, the winget offline copy and the confirmed state of the break detection – no package breaks. If packages are affected, a **preview** lists each package with old and new path; POCKET KNIFE first creates a **backup** automatically, and if anything goes wrong on the way, everything is rolled back (files and database).

What is blocked (fool-proofing):

Action	blocked if ...
Delete, overwrite	a package (also a disabled step or a nested package), a scheduled or running deployment, a tenant (its folder) or the winget offline copy uses the file or folder – the message names the packages
Rename/move a file **inside** a used package folder	this would silently change the package (e.g. an MST referenced in the arguments) – move the whole folder or adjust the package; the installation file itself may be renamed
Moving a central package into a tenant folder	that tenant's admins could change the files there (likewise: saving a central package with files from a tenant folder)
OU admin: uploading, creating folders, moving into a folder used by a central package or another tenant	otherwise e.g. a DLL could be placed next to someone else's installation file; if someone puts something there directly on the share, the break detection reports it as a **break**
Moving tenant packages out of their tenant folder	tenant packages may only use files from their folder
Names	characters such as `\ / : * ? " < > |`, reserved Windows names (`CON`, `NUL`, `COM1` ...), trailing dot or space
Links (junctions/symlinks) in the path	POCKET KNIFE never writes through a link – the file server would resolve it on its side

New files in a used package folder are allowed (e.g. an additional configuration); the package then shows "new in package folder" until someone accepts the new state. **Who uses this?** shows the packages for each entry, the column **Used by** is right in the list. Every action is logged (category "Repository") with user and tenant.

Limits: maximum file size (default 4096 MB) under **Settings → Repository & winget**; for OU admins additionally the allowed file types (MSI, MSP, MSU, MST, EXE, CMD/BAT, PS1, REG, ZIP, 7Z, CAB, MSIX/APPX, INI/INF, XML/JSON/TXT/CFG, certificates, ISO/WIM and similar) and a **storage quota** per tenant (on the tenant, empty = default from the settings, 0 = unlimited).

Tenants: OU admins see and edit only their tenant's folder; they cannot rename, move or delete the folder itself. If a tenant may build its own packages and has no repository folder yet, POCKET KNIFE creates `Mandanten\<Name>` itself – one folder per tenant, shared by all its admins; below it they create their own subfolders. Two tenants can never share a folder (not even nested). The repository and its server stay hidden from OU admins.

Restructure the repository

Tidy up a grown repository before introducing tenant folders – main admins, button **Restructure repository …** at the top right. Recommended structure: one folder for the central packages (e.g. `Zentral`) and `Mandanten` with one subfolder per tenant.

1. **New main folder** – type or pick; if it does not exist it is created.
2. **What goes in?** – tick the existing folders (**Select all** leaves out `Mandanten`).
3. **Preview** – every affected package with old and new path; problems (see table above) are listed at the top, then it cannot run.
4. **Run** – in one go: automatic backup first, everything is rolled back on an error.

Duplicate block

The same software should exist only once – either central or with exactly one tenant. When saving, copying and importing a package and when uploading a file, POCKET KNIFE compares:

- the **winget ID** (install or upgrade),
- the **MSI product code** (from the step or from the MSI file),
- the **fingerprint (SHA-256)** of the installation file – also detects renamed copies.

If the same software exists in **another scope** (central, tenant A, tenant B – also between two tenants), it is rejected, naming the existing package: please use that one – **everyone may deploy all packages**. Within the same scope it is allowed (e.g. two variants with different arguments). The name alone does not count. Main admins can override (**Save anyway** or **Upload anyway**); this is logged. A duplicate that already existed before 2.27 does not prevent saving other changes; if POCKET KNIFE finds it later (e.g. once the fingerprint is computed), a note is written to the log.

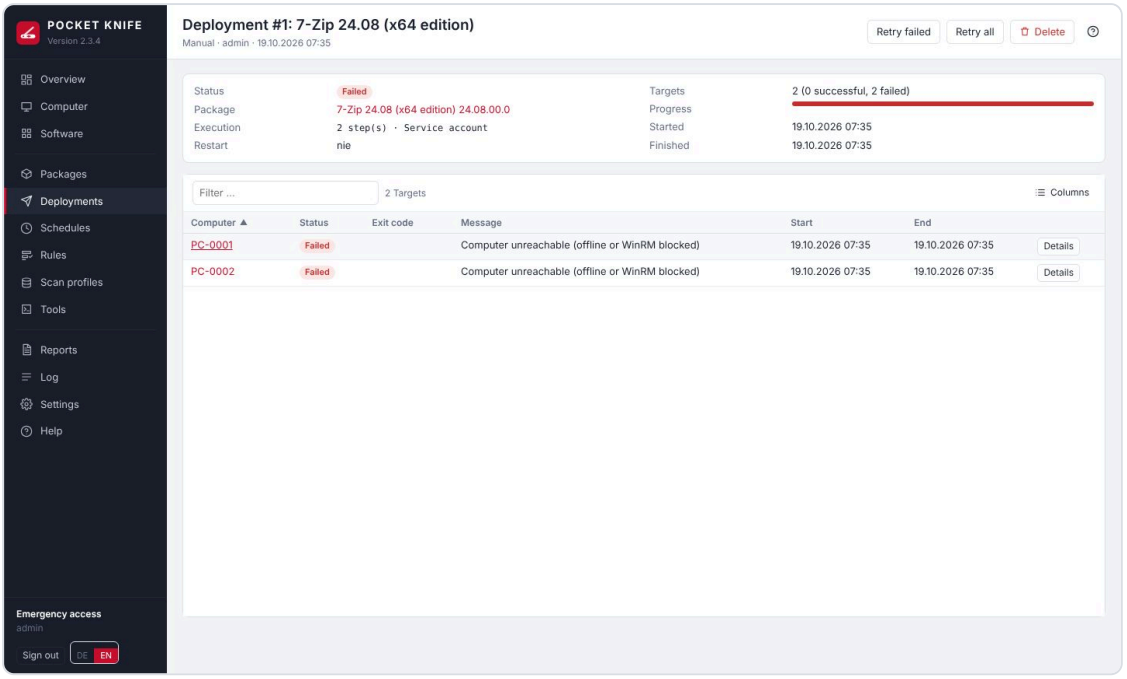
Find file (repair by fingerprint)

If something was moved or renamed outside POCKET KNIFE (e.g. in Windows Explorer), the package shows “file missing” or “folder missing”. **Find file …** in the package’s **Repository files** section searches the whole repository – candidates by size (from the in-memory index), confirmed by fingerprint; a folder counts as found when all its files are at the new location. **Apply paths** then adjusts all packages that use the old path (with a backup first). OU admins: only within their tenant folder. The index knows newly stored files after 5 minutes at the latest.

Deployments

Several packages at once: in **Packages**, tick several rows (or select with Ctrl/Shift) and click **“Deploy N packages …”** at the top right – or right-click → “Deploy N packages …”. In the dialog you can add further packages and change the order by dragging the entries. All packages run as **one** deployment: one after another per computer in this order, never two installations at the same time (otherwise Windows Installer likes to report 1618 “another installation is in progress”). Optionally the remaining packages are skipped for a computer after an error.

Deploy with the **Deploy …** button (computer list, package, software) or by right-clicking. The target is the selected computers or a collection; the time is immediate or scheduled.



Result of a deployment

The deployment shows the progress per computer. **Details** opens the result of each step with the exit code and the complete output. “Retry failed” repeats the whole package on the computers that did not succeed.

Retrying individual steps: If a step failed, the deployment shows the section **Failed steps** – per step with the affected computers and two buttons:

- **Only this step** – runs just this step again (e.g. a service start that failed because of a lock).
- **From this step** – runs this step and all following steps including the finish, i.e. also those that did not run after the error. Successful steps before it are not repeated.

For a single computer, do the same in **Details** via **Retry ▾** next to the failed step. Each retry creates a new deployment; package conditions, “Run as”, reboot and scan settings stay as in the original package.

Choose steps yourself: Choose steps … (at the top of the deployment) lists all steps of the package with their previous result (“2 OK”, “1 error”). Tick the steps – failed ones and those not run after a failure are preselected –, then choose the targets: unsuccessful, all or individual computers. A new deployment with only these steps is created, in the package’s order and with its conditions and “Run as”; steps disabled in the package run if chosen, finalization steps only if chosen. If the package has “Restart at the end”, the restart can be deselected for the retry (default “never” unless an install or restart step is chosen). OU admins can only do this for deployments that exclusively affect their own computers.

Status	Meaning
Successful	all steps succeeded
Skipped	a package condition does not apply (e.g. wrong operating system)
Failed	a step set to “Stop package” failed, or the computer was not reachable

Notification on the client

After a **successful installation or update**, the signed-in user sees a Windows notification – e.g. **“7-Zip wurde aktualisiert.”** / in English **“7-Zip has been updated.”** with **“Installed version: 26.00”** below and a green check mark, sender “Softwareverteilung” (configurable).

- Applies to manual deployments, schedules, winget updates and the kiosk.
- **Installed** or **updated**: winget upgrades and programs that were already in the inventory (in an older version) count as “updated”. For winget, POCKET KNIFE takes name and version from the output

(“Found 7-Zip … Version 26.00”), otherwise from the package name and version.

- **No notification** if nobody is signed in, if everything was already installed or up to date, for uninstallations and for pure script/registry packages.
- Several signed-in users (e.g. terminal server) each see the notification in their session.
- **Settings → Schedules & Performance → Notification on the client:** on/off, sender (a changed sender applies from the next notification), language (German/English), **Send test notification** to a computer. Per package under *Package properties → Notification*: as in the settings, always (also script packages) or never.

Technology: A short WinRM connection starts a scheduled task *as each signed-in user* (no password, no elevated rights, no visible window) – only this way does the notification appear in their session; as service account or SYSTEM nobody would see it. The display script is stored in `C:\Program Files\PocketKnife` (only administrators can write there); name and version are only passed as data, never as a command. Afterwards the script checks in the notification center whether Windows accepted the notification – if not, it shows it with the sender “Windows PowerShell”, which Windows always knows. If the user has switched off notifications from this sender in the Windows settings, that is respected (no fallback). The script is started with `-File` from “Program Files”, so it also runs under Software Restriction Policies (SRP), AppLocker or WDAC (constrained language mode) – as long as scripts from “Program Files” are allowed, as in the default rules. Only if a GPO sets the execution policy to AllSigned/Restricted does POCKET KNIFE start it via a short encoded command. The script reports its result itself; if there is no answer, the log says “display script did not start for the user”; POCKET KNIFE resets the folder permissions to the “Program Files” default with every notification. **The result per user is in the log** (“shown”, “shown with sender Windows PowerShell”, “not shown by Windows – notifications off/Do not disturb/policy” …), also for the test in the settings. The deployment does not wait for it.

> If no notification appears despite a successful test: check whether the user has notifications or “Do not disturb” turned on and whether a GPO disables notifications. As a last resort, turn on

> **Compatibility** (sender “Windows PowerShell”, which is always registered in Windows).

Message on restart and shutdown

Before POCKET KNIFE restarts or shuts down a computer, Windows shows the signed-in user a message. Its text is set under **Settings → Schedules & Performance → Message on restart and shutdown**:

- **Sender** – e.g. “Your IT department” instead of “POCKET KNIFE”.
- A separate text per occasion: **Restart** and **Shutdown** from the computer menu, **Restart after deployment**, package step **“Restart”** and **Restart from the kiosk**. Empty field = default text (German); below each field an example shows what the message will look like.
- Placeholders: `{absender}` (sender), `{sekunden}` (seconds), `{minuten}` (minutes), `{benutzer}` (kiosk only: who requested the restart). Example: `{absender}: Your computer will restart in {minuten} minutes. Please save your work.`
- A custom text in the package step “Restart” takes precedence (the same placeholders are allowed there).
- At most 400 characters; double quotes become single quotes (Windows limits the message to 512 characters).

POCKET KNIFE restarted during a deployment

If the container is restarted (update, Docker host reboot) while something is still running on computers, those targets are marked **“Interrupted (POCKET KNIFE was restarted)”** (failed – schedules catch them up as usual). POCKET KNIFE then cleans up on the computer by itself as soon as it is reachable (checked every 5 minutes):

- **If the installer is still running**, it is **not** killed but waited for – an installer stopped halfway does more damage than one that finishes. Only if it runs longer than the step’s time limit is it stopped (as with a normal timeout).
- Afterwards the working folders (`C:\Windows\Temp\PocketKnife\…` including copied installation files) and scheduled tasks (`\POCKETKNIFE\PK_…`) are removed.
- The target’s message shows the result, e.g. “– cleaned up on the computer (the interrupted step still finished, exit code 0)”; the log

(category “deploy”) lists what was removed.

- **New deployment to the same computer:** it cleans up first and **waits** while the old installer is still running (“Waiting: an installation from an interrupted run is still running on the computer …”) – so two installations never run at the same time (error 1618). Cancelling is possible meanwhile.
- If a computer stays unreachable for 14 days, POCKET KNIFE gives up; leftovers older than 2 days are removed by the next inventory scan anyway.

Tip: install updates preferably when no deployment is running (**Deployments** → status “running”).

Schedules

Schedules are the one place for everything that is distributed or updated automatically (the former “rules” have been merged into them and were converted during the update). When creating one, choose the **kind**:

Kind	What happens
Deploy packages	one or more saved packages, in a fixed order per computer
winget updates	update outdated software on the targets with `winget upgrade` – all outdated or only specific winget IDs, with exceptions; one deployment per program

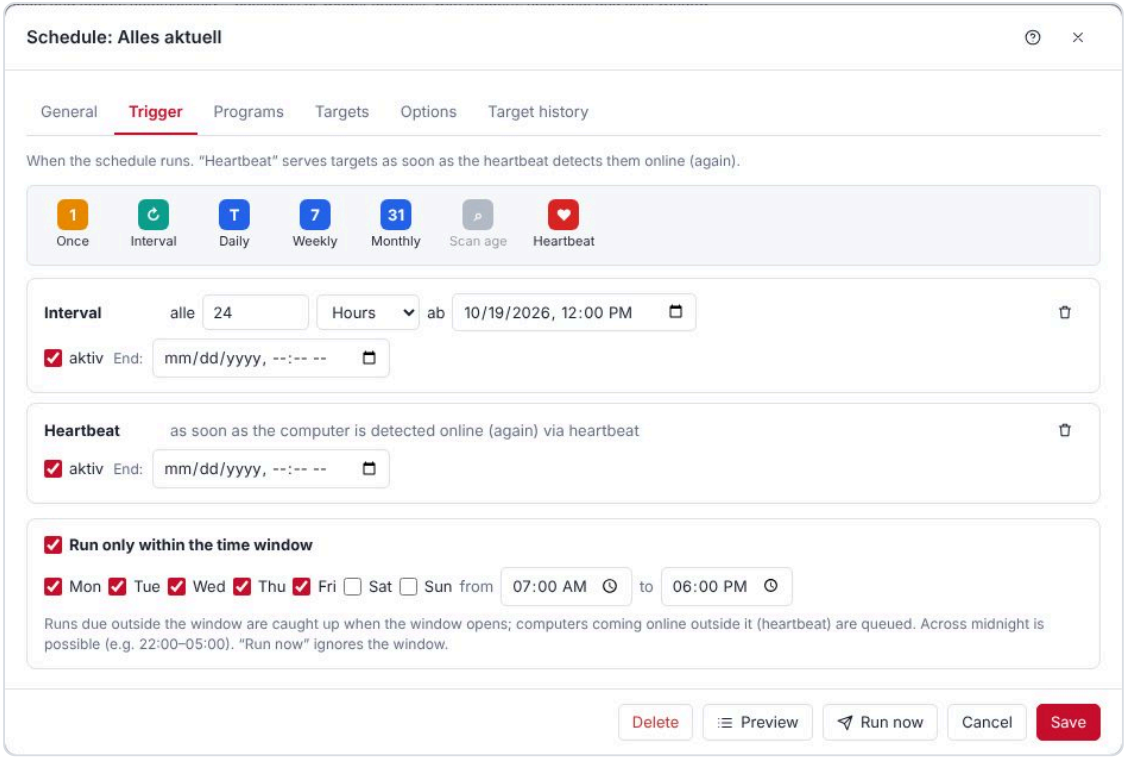
Common to both kinds:

- **Triggers:** Once, Interval, Daily, Weekly, Monthly and **Heartbeat**: serves every target as soon as it is reachable (WinRM answers) and has not been served yet – after power-on or Wake-on-LAN, and for a new schedule or new collection members at the next heartbeat (default every 5 minutes; a manual “Heartbeat” checks immediately). Targets already attempted are not retried continuously – use the wait after failure, other triggers or “Re-serve selected” for that. Without a trigger, the schedule only runs via “Run now”. Without a heartbeat trigger, only computers that were offline during an earlier run and were queued are served at power-on.
- **Time window** (optional): weekdays and time, also across midnight (e.g. Mon–Fri 10 PM–5 AM). Runs that become due outside the window are caught up when the window opens; computers coming online outside it are queued. “Run now” ignores the window.
- **Targets:** collections (re-evaluated on every run, including “All computers”) and individual computers.
- **Offline computers:** queue them and serve them as soon as they are online (heartbeat) – or skip them.
- **Wait after failure:** a failed computer (with winget: the same program) is retried after X hours at the earliest.
- **Catch up on failures** (default): failed computers are supplied again automatically without waiting for the next date – as soon as they are reachable and the wait time (at least 1 hour) has passed, within the

time window, at most “Max. failed attempts” times (three times if 0). If a run could not start at all (e.g. database briefly locked), it is retried every 15 minutes until it succeeds.

- **Target history:** status per computer, “Re-serve selected”, “Reset history”.
- **Preview:** shows in advance what a run would do now (with winget: which programs on which computers).

Only for **Deploy packages**: skip the remaining packages on error, do not re-serve successful targets, **re-serve after X hours** (e.g. combined with a dynamic collection “7-Zip missing or outdated” – the environment maintains itself), maximum failed attempts.



Schedule

Example – evening patch day: kind “Deploy packages”, trigger “Weekly, Wednesday 7:00 PM”, target “Workstations”, packages “Firefox”, “7-Zip”, “Adobe Reader”, queue offline computers. Computers that are off in the evening are served the next time they are turned on.

Example – always up to date: kind “winget updates”, all outdated, exception `Oracle.JavaRuntimeEnvironment`, triggers “Interval every 24 h” **and** “Heartbeat”, time window Mon–Fri 7 AM–6 PM, target “All computers”. Computers switched on during the day are updated right away, all others once a day.

Tools

Select computers → **Tools ▾** or right-click → Tools.

Built-in	
Ping & Ports	reachability and open ports
Remote Desktop / VNC in browser	remote session in a separate tab
Wake-on-LAN	via a reachable computer in the same subnet (relay)
Restart / Shut down / Cancel	with a 60-second warning
gpupdate /force	refresh group policies
Check / install winget	provides winget where it is missing
Scan with profile	immediate scan
Analyse disk space	folder sizes as a ring chart – see below
Files (Explorer)	manage the client's files in the browser – see below
Event log	Windows event logs in the browser – see below

Analyse disk space (Tools ▾ → *Analyse disk space …* or on the computer under **Disks**): shows where the space on a drive or in a folder goes – like TreeSize or WinDirStat, but without installing anything on the client.

- Choose a **path** (drives from the last scan are suggested, also folders like `D:\Data` or `%SystemDrive%\Users`) and a **depth** (3–6 levels), then **Analyse**. The client walks the file system as the service account via WinRM; a drive with a few 100,000 files usually takes less than a minute. The run continues on the server even if you leave the page.
- **Ring chart:** the centre shows the current folder and its size, the inner ring its content, the outer ring one level deeper (same hue, lighter). Grey: grouped small entries ("more folders/files") and files.
- **Zoom:** click a ring segment or a row of the list to open the folder; click the centre, "One level up" or the path at the top to go back. Folders below the chosen depth are only recorded as a total (▸ more) – one click analyses them specifically.
- **List** next to it with exact size, share and file count; right-click: analyse in detail, copy path. Hovering shows path, size and share.
- **Open in Explorer:** the folder icon at the end of each row, the button above the chart or the context menu jumps straight to **Files** – into the

folder, or for a file into its folder with the file selected. Clean up right there (delete, move, download).
- Results are saved (the last 8 per computer, selection at the top right).
- Linked folders (junctions, symbolic links) are not followed so that nothing is counted twice. Unreadable folders (e.g. *System Volume Information*) are counted and marked. Hard links (e.g. in `C:\Windows\WinSxS`) count at every occurrence, as in Explorer.

Custom tools (Tools → New tool) are PowerShell, commands (CMD) or URLs. Tools keep running on the server even if you close the browser.

Variables are replaced with the values of the respective computer before running (in PowerShell, command and URL):

Variable	Value	Example
`$(Computer)`	computer name (NetBIOS)	`PC-0815`
`$(Host)`	fully qualified DNS name; the computer name if unknown	`pc-0815.corp.local`
`$(IP)`	first IP address from the last scan	`192.168.10.25`
`$(MAC)`	first MAC address from the last scan (scanner “Network adapters”)	`00:11:22:33:44:55`
`$(User)`	last detected logged-on user (scan/heartbeat)	`CORP\jdoe`
`$(Domain)`	the computer’s domain from the scan, otherwise the domain from the settings	`CORP`

Case-sensitive; unknown values are replaced with an empty string. In a **URL** the values are URL-encoded; addresses with `javascript:`, `data:`, `vbscript:` or `blob:` are rejected.

In PowerShell tools also as variables: `$PK_Computer`, `$PK_Host`, `$PK_IP`, `$PK_MAC`, `$PK_User`, `$PK_Domain` are available safely quoted – even if a value contains apostrophes or `$` (e.g. a user name). For values that come from users (`$(User)`), prefer `$PK_User` over inserting `'$(User)'` into the text.

```
# Example: flush the DNS cache and reload group policies
ipconfig /flushdns
gpupdate /target:computer /force
```

Files (Explorer)

Tools ▾ → Files (Explorer) … or on the computer under **Disks**. Access the client's drives via the administrative shares (`\\PC\C$`), without installing anything on the client – the service account is a local administrator there.

- **Browse:** choose a drive (from the last scan) or enter a path at the top; double-click opens folders, the path at the top goes back.
- **Search:** in the open folder, enter a name or pattern in the search field above the list (`setup`, `*.log`) and press **Search** or Enter – the folder and all subfolders are searched (the field does not filter while typing). The search runs **on the client** itself (via WinRM, also with software restriction/AppLocker in Constrained Language Mode); only the matches come back, nothing is stored. Windows filters the names itself – even a whole drive with all subfolders is usually searched within seconds to a few minutes (at most 5 minutes, 300 matches). Matches behind links do not appear twice, unreadable entries are skipped (the number is shown at the top). Clicking a match opens its folder, for a file with the file selected; right-click also offers **Download** and **Copy path**. **End search** or Esc returns to the folder. If WinRM is not reachable, the search runs more slowly via the share (at most 20 seconds).
- **Manage:** right-click entries – **Download** (file), **Copy**/**Cut** and **Paste** in the target folder (on the same computer, whole folders too), **Rename**, **Delete** (multi-select; folders with their content). **New folder** and **Upload** in the toolbar.
- Links (junctions/symlinks), hidden and locked entries are marked. Everything runs as the service account and is logged. Deleting and overwriting cannot be undone.

Links (junctions/symlinks, e.g. `C:\Users\All Users` → `C:\ProgramData`) are marked with ↗. Opening jumps to the target, deleting removes only the link, not the target.

Event log

Tools ▾ → Event log …: reads the Windows event logs (System, Application, Security, Setup, Defender) live over WinRM. Filter by level (warning and worse, errors, critical only), search by text/event ID/source, choose the count; click a row for the full message.

Software Kiosk

In the kiosk, users install approved software themselves – without administrator rights and without calling IT. It is a separate, simple page: `https://<pocketknife>:8443/kiosk` (e.g. as a desktop shortcut via GPO). Users sign in with their normal Windows account – with user name and password or, if set up, automatically via Sign-in with Windows (SSO).

Setting up under **Kiosk** (navigation):

1. **Settings:** turn the kiosk on and define who may use it – **AD groups** (name or DN, also nested, e.g. "Domain Users") and/or **OUs** (including all sub-OUs). Either one is enough. **"Choose from AD …"** opens an expandable AD tree where you tick entries; groups can also be searched by name. Optional: mail to IT when a request is waiting for approval; maximum number of requests per user in 24 hours (default 10); Sign-in with Windows (SSO) (default off).
2. **Catalog → Add software:** choose an existing package and define
   - **Approval:** install immediately or only after approval by an administrator
   - **Icon** (PNG, JPEG, GIF, WebP or ICO, at most 512 KB, ideally 128 × 128 pixels; SVG is not accepted for security reasons) – without an icon the kiosk shows the initials
   - **Category** and **description** for users
   - **Uninstall:** a package that removes the software again (e.g. right-click in "Software" → create uninstall package) – without it the kiosk shows no "Uninstall" button
   - **Only for:** only computers in a collection see the software (e.g. licenses of one department)
   - **Only for groups:** only members of these AD groups (name or DN, nested groups count; "Choose from AD …") see and order the software – e.g. a graphics program only for the design team. Empty = everyone who may use the kiosk. Applies in addition to "Only for"; POCKET KNIFE checks when showing the catalog and when ordering (result cached for 5 minutes). If AD is not reachable at the moment, such software stays hidden.
3. **Requests: approve** or **reject** waiting requests (with a reason the user sees); history of all requests with a link to the deployment.

What users see and may do:

- **Only their own computer:** POCKET KNIFE checks live via WinRM which computer the user is signed in to (using the logon data from the inventory, the browser's address and the DNS name). Nothing can be requested for other computers – not even via the API.
- **Only the catalog:** install or request, uninstall (if configured), status per software ("Waiting for approval", "Running", "Done", "Failed"), withdraw own requests as long as they are not running yet.
- **Plan a restart:** if an installation needs a restart (or the last scan reports a pending restart), the kiosk offers "Now (in 2 minutes)", "In 1 hour" or a time on the same day. A notice appears on the computer before the restart; a planned restart can be cancelled. Not possible for **servers** (including terminal servers) and while **other users** are still signed in to the computer.
- **No administration:** kiosk sign-ins have no access whatsoever to the POCKET KNIFE interface (nor to remote sessions). Members of the access group can also use the kiosk for their own computer.

Installations from the kiosk are normal deployments (name "Kiosk: ...") – with log, run as service account and a scan afterwards as usual. Everything is logged under the category "Kiosk". The sign-out when idle also applies in the kiosk.

Sign-in with Windows (SSO)

Optional – **off by default**. When turned on, the kiosk signs users in on domain PCs **automatically with their Windows account** (Kerberos, "single sign-on"): no user name, no password. The sign-in form remains as a fallback, e.g. for devices outside the domain.

Windows sign-in applies **only to the kiosk**. Members of the access group also get only the kiosk this way, never the administration – user name and password remain mandatory there.

How it works: The browser requests a Kerberos ticket for the kiosk's name (`HTTP/pocketknife.firma.local`) from the DC and sends it along. POCKET KNIFE verifies the ticket with a **keytab** – the key of the account that holds this name (SPN). POCKET KNIFE needs no connection to the

DC for this, and no password is transmitted. Who may use the kiosk is then checked exactly as for password sign-in (kiosk groups and OUs, via LDAP with the service account).

Requirements

- The kiosk is opened via a **fixed DNS name**, not the IP address – with an HTTPS certificate for that name (see Your own certificate).
- The clocks of the Docker host and the DCs differ by 5 minutes at most (NTP on the Docker host).
- The realm is `PK_DOMAIN` in upper case (if different: `PK_KERBEROS_REALM` in `.env`).

Setup step by step

In the example the domain is `firma.local` (NetBIOS `FIRMA`), the kiosk `pocketknife.firma.local`.

1. **DNS:** create an **A record** `pocketknife.firma.local` → IP of the Docker host. No CNAME – otherwise browsers request a ticket for the target name.

2. **Create an account for the name:** a new, normal user account, e.g. `svc-pk-http` – **without any rights**, "Password never expires", and under *Account → Account options* enable **"This account supports Kerberos AES 256 bit encryption"**. Do not use the service account: `ktpass` sets a new password in the next step.

3. **Register the SPN** (as domain admin):

   ```
   setspn -S HTTP/pocketknife.firma.local FIRMA\svc-pk-http
   setspn -Q HTTP/pocketknife.firma.local
   ```

   `-Q` must show exactly **one** account – a duplicate SPN breaks Kerberos.

4. **Create the keytab** (on a DC):

   ```
   ktpass -princ HTTP/pocketknife.firma.local@FIRMA.LOCAL -mapuser
   FIRMA\svc-pk-http -crypto AES256-SHA1 -ptype KRB5_NT_PRINCIPAL -pass * -
   out pocketknife-http.keytab
   ```

   The realm after `@` is in **upper case**. `-pass *` asks for a new password and sets it on the account. The keytab is as secret as a password.

5. **In POCKET KNIFE: Kiosk → Settings → Windows sign-in** → choose the keytab file → **Upload**. Tick “Offer Windows sign-in” → **Save** → **Test**. The test checks the keytab (name, realm, encryption), loads the key and resolves the name in DNS. Afterwards delete the keytab file on the DC or lock it away safely.

6. **Allow the browsers (GPO).** Browsers only send a ticket to sites they consider intranet:

   - **Edge and Chrome:** *Computer Configuration → Administrative Templates → Windows Components → Internet Explorer → Internet Control Panel → Security Page → Site to Zone Assignment List*: `https://pocketknife.firma.local` = `1` (Local intranet). In addition the policy **“AuthServerAllowlist”** = `pocketknife.firma.local` (Edge: *HTTP authentication → Configure list of allowed authentication servers*; Chrome likewise in the Chrome templates).
   - **Firefox:** policy **Authentication → SPNEGO** = `https://pocketknife.firma.local` (or `about:config` → `network.negotiate-auth.trusted-uris`).

7. **Shortcut** for users via GPO to `https://pocketknife.firma.local:8443/kiosk` – exactly this name, not the IP.

How the kiosk behaves

- When opened, the kiosk tries Windows sign-in **once automatically**. If it works, the user is signed in right away.
- If it does not, the form appears with a hint; **“Sign in with Windows”** tries again.
- After **signing out** or the **sign-out due to inactivity**, the kiosk does not sign in again by itself in this tab – use the button.
- If the kiosk is opened via a different name or the IP, it shows the correct address as a link.
- Disabled accounts are rejected. Running sessions are re-checked in AD automatically as usual (every 2 minutes).
- The log (category “auth”) shows “Sign-in … (kiosk, Windows sign-in user@REALM)” or the reason why it failed.

Security: The keytab is stored **encrypted** in the database (like the passwords; included in every backup, readable only with `pocketknife.key`). For verification it is decrypted into memory only (`/tmp` is a tmpfs in the container). It can never be retrieved via `/api/settings`. “Remove” deletes it immediately.

Troubleshooting

Observation	Cause and solution
The browser shows its **own sign-in window**	The site is not allowed as intranet → check the GPO from step 6 (`gpupdate /force`, restart the browser). “Cancel” leads to the normal form.
“The browser did **not send a Kerberos ticket**”	Browser not allowed, or opened via IP/another name.
“The browser sent **NTLM instead of Kerberos**”	No ticket for the name: SPN missing or duplicate (`setspn -Q`), opened via IP or CNAME. Check on the client: `klist get HTTP/pocketknife.firma.local`.
“The ticket **does not match the keytab**”	The password of `svc-pk-http` was changed after `ktpass`, or AES is not enabled for the account → create and upload the keytab again. Then run `klist purge` on the client.
“The **clocks** … differ too much”	Set up NTP on the Docker host.
“No access: you are not in any kiosk group …”	The ticket is fine, but the user is in no kiosk group/OU.
Does not work behind a **reverse proxy**	The proxy must pass the `Authorization` header and the `WWW-Authenticate` response through unchanged.

Remote Sessions

Computer → **Remote Desktop** (or Tools → VNC in browser) opens the session in a separate tab – no plugin required. You can sign in with your own POCKET KNIFE credentials (the password is kept only encrypted in memory), with different credentials, or on the computer itself. For **VNC** there are “Without password / server prompts”, “With password” and – if stored under **Settings → Remote sessions** – the **central VNC password** (stored encrypted and then preselected in the dialog; classic VNC only uses the first 8 characters). Within the session: Ctrl+Alt+Del, Windows key, clipboard, full screen, fit to window. This requires the `pocketknife-guacd` container, which starts automatically with `docker compose`.

Reports and Log

Reports provide lists such as installed software, outdated software, operating systems, disk space, Windows updates, security status, local administrators, software changes or the deployment history – as a table, CSV (Excel-compatible) or print view.

The **Log** contains all events (sign-ins, scans, deployments, changes) and can be filtered by level, area and text. In addition, POCKET KNIFE writes a rotating file to `/data/logs/`.

Custom reports

Under **Reports → Custom reports → New report …** you build reports from all collected data yourself – with the same fields as dynamic collections.

Tab	Contents
Data	**Template** (e.g. hardware overview, software per computer, count software, drives below 15 % free, Windows 10 still in use, Defender off) as a starting point, name, description and the **data source**: what a row stands for – a computer, an application, a drive, a printer, a service, a hotfix, a local user, an entry of a custom scanner …
Columns	any columns of the computer and the data source, order via ↑/↓, custom label possible
Filter	**which rows** (e.g. only applications with "Adobe" in the name, only drives below 20 GB free) and **which computers** (as for collections, with All/Any/Not all/None groups), optionally only one collection
Output	**Summarise**: count identical rows (e.g. applications by name + version → "Number of computers"), sorting, maximum rows
Preview	runs the report with the current input (first 200 rows) without saving

Custom reports appear in the report list like the built-in ones – with collection selection, CSV, print and **By e-mail …** (also in a **report e-mail** together with other reports). **Notification …** turns a report into a

notification: an e-mail as soon as the report returns rows. Each row is reported only once; if only numbers or dates of a row change (e.g. free space), it counts as the same case. Checks run at an adjustable interval (default 15 minutes). Deleting a custom report removes it from report e-mails and deletes its notification.

Examples: “Computers without BitLocker on C:” (source Computer, filter), “Who has software X in which version” (source Applications, row filter name), “Printers per site” (source Printers, columns AD site + printer, summarised), “Profile sizes over 10 GB” (source custom PowerShell scanner).

Reports by e-mail

Under **Reports → E-mail delivery** (or with **By e-mail …** directly from a report) you combine one or more reports into a **report e-mail**:

- **Reports:** any number of existing reports with their parameters (collection, limits …), order via ↑/↓. All of them go into **one** e-mail.
- **Schedule:** once, interval, daily, weekly or monthly – as for scan profiles. Without a trigger the e-mail is only sent with **Send now …** (which can also send to a single test address).
- **Recipients & content:** recipients (empty = default recipients), an introduction, how many rows per report appear as a table in the body (0 = attachment only), a **CSV attachment** per report (Excel-compatible) and **“don’t send if all reports are empty”** – e.g. for “Low disk space”: an e-mail only when there is something to do.
- **Preview** shows the e-mail exactly as it will be sent.

An SMTP server must be set up (see below). Sending runs in the background; the result is shown in the list and in the log (category **E-mail**).

Notifications

Under **Settings → E-mail & notifications** you can turn on e-mails for certain events – each type separately; all are **off** by default:

Notification	When
Low disk space	drive below a GB and/or percent limit (optionally only certain drives, only one collection). Value from the last inventory scan.

Notification	When
Computer offline for a long time	not seen by the heartbeat for at least N days (computers disabled/removed in AD don't count)
Scan failed	the last inventory scan of a computer failed
Deployment failed	a deployment failed completely (optionally also partially) – one notification per deployment
Service account guard triggered	automatic access was paused because the service account was rejected
Backup failed	the automatic database backup failed
Emergency access used	every sign-in via the emergency access and every failed attempt for it
Custom report: …	a custom report returns rows – one notification per custom report

Checks run every minute. Each case (e.g. "PC-17 drive C:") is reported **only once**; several new cases are combined in one e-mail. Optional: a **reminder** after N hours while the case is open, and an **all-clear** when it is resolved. For disk space a case only counts as resolved once at least 10 % more than the limit is free again – no back and forth around the limit. **What would be reported now?** in the dialog shows the currently matching cases without sending anything. Events (deployments, backups) only count from the moment the notification is turned on.

Setting up the SMTP server

Settings → E-mail & notifications → SMTP server:

Field	Note
SMTP server, port	e.g. `mail.company.local` or Exchange/Microsoft 365 (`smtp.office365.com`, port 587)
Encryption	**STARTTLS** (587), **SSL/TLS** (465) or unencrypted (25, internal relay only)
Verify certificate	leave on; only turn off for internal servers with a self-signed certificate
User, password	empty = no login (relay by IP). The password is stored encrypted (`pocketknife.key`).
Sender, sender name	the sender address must be allowed by the server
Default recipients	for reports and notifications without their own recipients
Subject prefix	default `[POCKET KNIFE]` – handy for mail rules

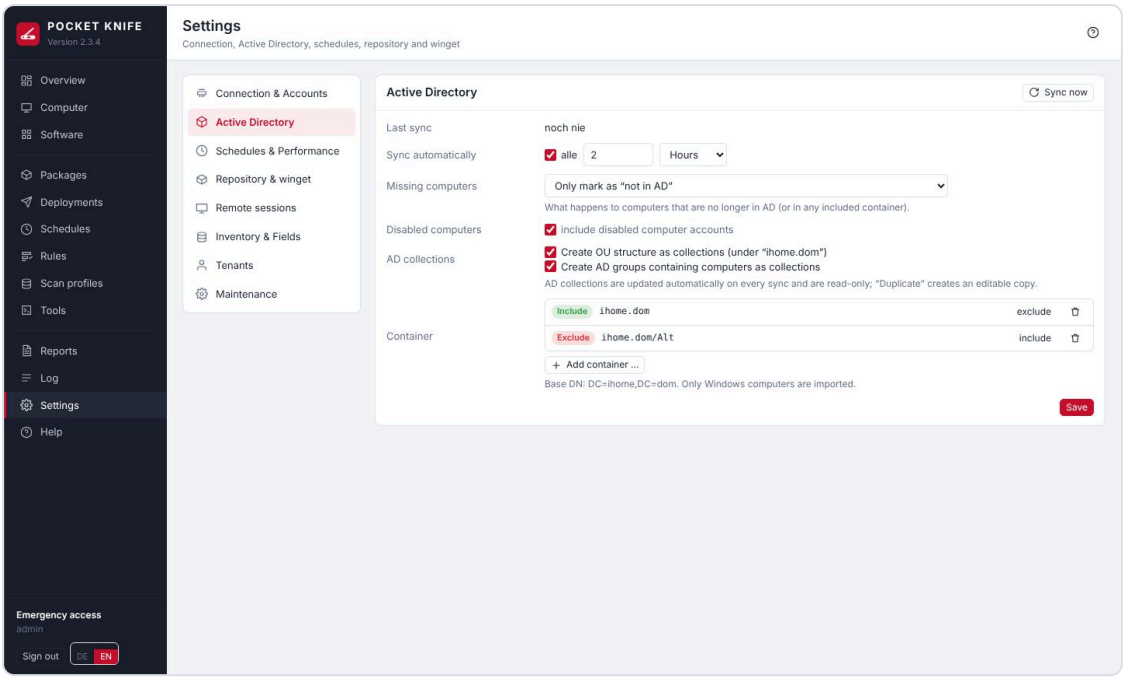
Send test e-mail saves the values and sends a test message; errors (login, certificate, port) are shown in plain language. The container needs network access to the SMTP port. Flood protection: at most 120 e-mails per hour.

Sending cleanly without being flagged as spam: The most reliable setup is logging in to your own mail server or Microsoft 365 (port 587, STARTTLS, verify certificate) with a dedicated mailbox, e.g. `pocketknife@company.com`, and **exactly this address as the sender**. Your mail server then signs and sends it like any normal e-mail (SPF/DKIM/DMARC match). POCKET KNIFE never sends directly to the internet; reputation and blacklists only concern your mail server, and the volume is low (at most 120 e-mails per hour).

E-mail "sent" but nothing arrives? After the test e-mail POCKET KNIFE shows the full SMTP dialogue and the server's last reply (usually with a queue ID, e.g. `250 2.0.0 Ok: queued as 4F3A2`). If the server replied like that, it took over the e-mail – delivery is then up to the mail server. Common causes: spam folder/quarantine, sender address doesn't exist (non-delivery reports go nowhere), relaying to external recipients not allowed, SPF/DMARC of the sender domain don't match, an anonymous relay discards mail from unknown hosts. With the queue ID or Message-ID the e-mail can be traced in the mail server log (Exchange: message tracking). POCKET KNIFE identifies itself to the server as `PK_PUBLIC_HOSTNAME.PK_DOMAIN` (e.g. `pocketknife.company.local`).

Settings

The settings are divided into sections (list on the left).



Settings – Active Directory

Section	Contents
Connection & Accounts	domain data, Test LDAPS, **sign out when idle**, **service account** and optionally a repository account (verified before saving, stored encrypted)
Active Directory	include/exclude containers (the closest rule wins), sync interval, handling of removed and disabled computers (deleted only once a computer has been missing from AD **for 7 days**; if no container is included or suddenly almost all computers are missing, nothing is changed), AD collections
Schedules & Performance	heartbeat, parallel scans/installations, timeouts, retention of log and history, notification on the client, message on restart
Repository & winget	repository path and test, winget index, default scope, automatic installation of winget, offline sources, proxy server
Remote sessions	guacd status, own credentials, keyboard layout, active sessions
Backup	automatic backup (daily and before updates), versions to keep, back up now, download

Section	Contents
E-mail & notifications	SMTP server with test e-mail, notifications on events (see Notifications)
Certificate (HTTPS)	web interface certificate: view, upload your own (PFX or PEM), create a request (CSR), backups, regenerate self-signed
Inventory & Fields	scan profiles, custom fields (e.g. asset number, room) – can also be created directly on a computer under **Custom fields**; free-text **notes** per computer are there too (stored only in POCKET KNIFE, never overwritten by the computer)
Tenants	tenants with their own collection, repository folder and optional OU admins
Maintenance	status, database size and "Clean up now", immediate AD sync and heartbeat, connection test, appearance, language, diagnostics package

Tenants: Packages and schedules can be assigned to a tenant. If the tenant has a collection, its packages are distributed only to the computers in that collection. If it has a repository folder, its packages may only use files from that folder (checked on save).

OU admins (tenant admins)

If a colleague should manage only part of the domain, for example the OU Hamburg, make them an **OU admin**:

1. Create a group in AD, e.g. `PK-OU-Hamburg`, and add the colleagues. They do **not** belong in the main admins group (`POCKETKNIFE-Admins`).
2. **Settings → Tenants → New tenant**: under **Computers** choose the OU's collection (an automatically created AD collection or a dynamic collection with the filter "Distinguished Name / OU"), under **OU admins** enter the group (**Choose …** searches AD).
3. Optionally enable **"may create their own packages and schedules"**. POCKET KNIFE then creates the **repository folder** itself (`Mandanten\Hamburg`) – or enter your own, e.g. `Kunden\Hamburg`. Optionally a **storage quota** in MB.

Members of the group then sign in as usual. "OU admin: Hamburg" is shown at the bottom left.

	Main admins (`POCKETKNIFE-Admins`)	OU admins
Computers, inventory, software, log	all	only the tenant's computers
Actions on computers (services, files, remote session, LAPS/BitLocker, run tools)	all	own computers only
Packages (central and those of other tenants)	change and deploy	view **all** and deploy to own computers; change only their own (copy: only if the software does not exist yet – duplicate block)
Repository	everything	only the tenant's folder: upload, folders, rename, move, delete (see Repository)
Own packages and schedules	–	only if enabled for the tenant; files only from its repository folder; command/PowerShell steps only with a separate approval (see Securing tenants)
Deployments	all	only with own computers; cancel/retry only if all targets are their own
Settings, scan profiles, kiosk administration, reports, e-mail, creating tools	yes	no

Security: The server checks the limits on **every** request, not just the user interface. For OU admins every function is blocked unless explicitly enabled; foreign computers are treated as non-existent. Anyone in both groups is a main admin. The emergency access is always a main admin. Memberships are checked at sign-in and then automatically every 2 minutes in the background – anyone removed from the group in AD loses access without further action; a change to the tenant in POCKET KNIFE takes effect immediately. POCKET KNIFE learns which computers belong to the OU during the AD sync (Settings → Active Directory, hourly by default; "AD sync now" under Maintenance). A tenant with OU admins always needs a collection (without one there would be no computers – never "all").

Further safeguards:

- The group is resolved **unambiguously** in AD when saving (first via the group name "sAMAccountName", which is unique across AD) and

stored with its **SID** – AD must be reachable for this. At sign-in only this group counts: a group with the same name in another OU, or one deleted and recreated, never grants access; renaming or moving does no harm. If a name is ambiguous, enter the full DN.

- Tenant packages (also those saved by main admins) and custom commands of OU admins never run as the service account but as "Local System" (or the signed-in user). Uninstalls using the command from the computer's registry never run as the service account for anyone – main admins included – because whoever is administrator on the computer determines that command. Repository files only with a repository folder for the tenant.
- Schedules of OU admins: only their own tenant's collection as a collection target. A tenant that once had OU admins reaches no computers without a collection; tenants with schedules can only be deleted once these are reassigned. Any change to a tenant's group, collection or permissions ends its OU admins' sessions (they sign in again).
- Remembering uninstall commands "for all", the central VNC password and the kiosk via the OU admin sign-in remain reserved for main admins or kiosk users.

> **Recommendation:** OU admins are administrators on their computers. POCKET KNIFE signs in there with the domain-wide service account. Therefore: use Kerberos instead of NTLM (Settings → Connection), add the service account to the "Protected Users" group (then "Kerberos only" – see WinRM by Group Policy) and enforce SMB signing via GPO. For even stricter separation, use a separate POCKET KNIFE instance with its own service account per site.

Securing tenants

A tenant should only bring to its computers what is in **its repository folder**. POCKET KNIFE enforces this in three levels:

1. Files only from its own folder. For tenant packages, Install and File copy take files only from the tenant's repository folder (`Mandanten\<Name>`). The server rejects other folders, UNC paths (`\\server\…`) and `..` when saving.

2. No own code without approval. Under “own packages”, the tenant has the additional switch **“may write command and PowerShell steps” – off** by default. Then, for OU admins:

Allowed	Not allowed
Install (MSI, EXE, MSU, **also .ps1/.cmd as a file from their own repository folder**), winget, File copy, Registry, Uninstall (with the detected command), Restart, Message, Wait, Scan, nested packages	“Command” and “PowerShell” steps, custom uninstall commands (in packages and in the software list)

Scripts therefore belong **in the repository as files** – there the integrity check (fingerprint) sees and checks them like any other installation file. Steps written by a main admin may be left unchanged by an OU admin, but not changed or added. Existing packages keep running.

3. What a tenant package reaches on the network. Tenant packages always run on the client as **Local System** – never as the service account, even if a main admin saved them. If a script or installer accesses a share, it signs in there with the **computer account** (e.g. `CORP\PC-17$`). It therefore only reaches what is shared with “Domain Computers”, “Authenticated Users” or “Everyone”.

> **Important:** every installer is executable code – an MSI can also access shares or the internet in the background. Levels 1 and 2 determine **where** the code comes from; what it **reaches** is determined by the permissions on the network (level 3). Therefore also:

- **Check shares:** anything not meant for all computers should not be shared with “Domain Computers”, “Authenticated Users” or “Everyone” – on the share **and** in NTFS. The repository does **not** need any permissions for “Domain Computers” either: POCKET KNIFE reads it with the repository/service account and copies the files to the client itself – otherwise a tenant script could read other tenants’ files.
- **Optionally restrict by GPO** what clients can reach: *Computer Configuration → Policies → Windows Settings → Security Settings → Windows Defender Firewall with Advanced Security → Outbound Rules* – allow outbound SMB (TCP 445) only to the file servers the clients need (e.g. logon scripts, profiles, print servers). Try it in a test OU first: domain controllers (SYSVOL/NETLOGON) must always remain reachable.

- **Notice when saving:** if a tenant package contains access to shares (`\\server\…`, `net use`, `New-PSDrive`) or downloads (`Invoke-WebRequest`, `curl`, `bitsadmin`, `https://` …) in commands, PowerShell or arguments, POCKET KNIFE shows a notice after saving and writes it to the log. This is **a help for reviewing, not a block** – such things can be rewritten. Script files in the repository are not searched.

Your own certificate

POCKET KNIFE serves HTTPS itself – no reverse proxy is needed. A self-signed certificate is generated on first start; replace it under **Settings → Certificate (HTTPS)**. A new certificate is **active immediately** (no restart); the previous one is backed up.

Option A – upload a finished certificate

- **PFX / PKCS#12** (`.pfx`, `.p12`) with password – contains certificate, key and usually the chain. Easiest from the Windows certificate manager: *Export → Yes, export the private key*.
- **PEM / CER** (`.crt`, `.cer`, `.pem`, DER or `.p7b`) plus key file (`.key`, encrypted keys are fine). Intermediate certificates optionally as a separate file.

Check shows in advance what would be installed. Rejected are: key does not match, expired or not yet valid, not for server authentication, RSA below 2048 bits. You get a note when `PK_PUBLIC_HOSTNAME` is not in the certificate, intermediates are missing or it expires in less than 30 days.

Option B – request (CSR) for your own certification authority

1. Enter the names (CN and every address used to open POCKET KNIFE) → **Create request**. The key is generated inside POCKET KNIFE and never leaves it.
2. Download the CSR and submit it, e.g. to Active Directory Certificate Services:

```
certreq -submit -attrib "CertificateTemplate:WebServer" pocketknife.csr
pocketknife.cer
```

3. Upload the issued certificate using the **PEM / CER** format – without a key file; the key from the request is used.

> For browsers to trust the certificate without a warning, the CA's root certificate must be trusted on the workstations (automatic in an AD environment with Certificate Services). **Download certificate** exports the current certificate, e.g. for a GPO.

Reset: Restore one of the last five backups or create a new self-signed certificate (e.g. after changing `PK_PUBLIC_HOSTNAME`). The files are in `/data/tls` (`server.crt` with chain, `server.key`); you can still place them there by hand and restart the container.

Kerberos instead of NTLM

POCKET KNIFE signs in with the service account via **WinRM** (PowerShell) and **SMB** (package copies, repository). The default is **NTLM**. Microsoft has deprecated NTLM; **Kerberos** is the future-proof option. From version 2.6, Kerberos is included in the container and can be tested and switched without a restart.

Not affected: LDAPS (simple bind over TLS, no NTLM) and the web interface sign-in. **Remote sessions via RDP** (guacd/FreeRDP with NLA) still use NTLM.

Method	Behaviour
NTLM (as before)	WinRM and SMB via NTLM only
Negotiate	Kerberos where possible – otherwise NTLM automatically (e.g. when connecting by IP address)
Kerberos only	no NTLM fallback; anything that cannot use Kerberos fails with a hint

Requirements

- Computers with a **fully qualified DNS name** (e.g. `pc-0815.corp.local`, from the AD sync). Kerberos does not work with IP addresses. Short names are completed with `PK_DOMAIN`.
- **Time**: the Docker host and the domain controllers must be within 5 minutes (NTP on the Docker host).
- The Docker host reaches the domain controllers on **port 88** (TCP). DCs are found via DNS (`_kerberos._tcp.<domain>`); otherwise set `PK_KDC=dc1.corp.local,dc2.corp.local` in the `.env`.
- **Repository path** with a DNS name, e.g. `\\fileserver.corp.local\software$` instead of

`\\fileserver\software$` or an IP.

- The service account needs nothing special. On “encryption type not supported”: enable “This account supports Kerberos AES 256 bit encryption” in AD and set the password once more.
- SPNs do **not** have to be created: every domain computer has `HOST/<name>`, which `HTTP` (WinRM) and `cifs` (SMB) map to – Samba AD included.

Switching step by step

1. Update to version 2.6 or later (the image is rebuilt).
2. **Settings → Connection & accounts → Authentication**: choose a computer, “Kerberos only”, **Test**. All steps green and “The client reports: sign-in via Kerberos” = ready. The test does not change the active setting.
3. Set the method to **Negotiate** and save – effective immediately, no restart.
4. Watch for a few days (scans, deployments, repository). On a client, Event Viewer (Security, event 4624) shows “Authentication Package: Kerberos” for the service account.
5. Switch to **Kerberos only**. To keep it in the configuration permanently: `PK_WINRM_AUTH=kerberos` in the `.env` and “as .env” in the interface.
6. Later, NTLM can be restricted for the service account via GPO (“Network security: Restrict NTLM” – audit mode first).

Back to NTLM: choose “NTLM” in the same list and save.

Typical messages in the test

Message	Solution
Kerberos libraries missing	pull or rebuild the image (`sh update.sh --pull` or `update.sh` with the ZIP file)
No KDC found / not reachable	check DNS SRV records or set `PK_KDC`; firewall port 88
Clock skew	set up NTP on the Docker host
No service principal name (SPN)	address the computer by DNS name, check DNS record and computer account
Client reports NTLM instead of Kerberos	name is not recognised as a domain computer – check DNS name

Message	Solution
Message stream modified	The device at the address cannot read the ticket: the DNS record points to another device (stale address, duplicate name) or the computer's trust relationship is broken (`Test-ComputerSecureChannel -Repair` on the client). The computer appears with a yellow dot "not confirmed".

Custom Kerberos configuration: POCKET KNIFE generates the `krb5.conf` from `PK_DOMAIN` (realm in upper case; if different: `PK_KERBEROS_REALM`). A custom file `/data/krb5.conf` takes precedence.

Diagnostics package

Under **Settings → Maintenance → Diagnostics package**, POCKET KNIFE creates a ZIP with the log, statistics and settings of the last days (since the last update or 1–30 days, default 7). This makes it possible to trace errors and anomalies without access to the environment – e.g. to pass on to the developer.

Contents

File	Contents
`versionen.json`, `aenderungen.md`, `behobene_fehler.txt`	version history, changes since the oldest version in the period, fixed errors
`LIESMICH.txt`	period, version, whether anonymized
`umgebung.json`	version, runtime environment, number of computers/packages/collections, database size, state of the schedulers
`einstellungen.json`	settings without passwords, keys and keytab
`statistik.json`	success rate per package, exit codes, most frequent errors, scan duration, warnings, kiosk requests
`protokoll.csv`, `fehler.txt`	log of the period, warnings and errors separately
`bereitstellungen.csv`, `fehlgeschlagen.json`	deployments; for failures the end of the output (at most 3000 characters per step)
`scans.csv`	scan runs with duration and result

Passwords, the vault, sessions, keytab and certificates are **never** included – not even without anonymization. Passwords in command lines of steps and outputs (e.g. `PASSWORD=…`, `net use … /user:account password`, `ConvertTo-SecureString '…'`) are replaced by `***`.

Versions: Every line in the package carries the version under which it was created. `statistik.json` separates the whole period from the **running version** and shows warnings, errors and results per version. In addition there are `versionen.json` (since when which version ran), `aenderungen.md` (changes from the oldest version in the period up to today) and `behobene_fehler.txt` (errors the self-test checks again with every version). An error that only occurred under an older version can thus be recognized as probably fixed. After an update, the period **Since the last update** is recommended; the card shows since when the running version has been running. The version history is kept since 2.22; earlier updates are added from the start messages in the log – even older entries appear as “vor 2.x”.

Anonymize (on by default): names are replaced by fixed placeholders – computers `PC-001`, users `USER-001`, servers `SRV-001`, persons `Person-001`, IP addresses `10.99.x.y`, MAC addresses `02:00:…`, mail addresses `user01@firma.local`, domain `firma.local` / `FIRMA`, OUs and groups as well. The same name gets the same placeholder in every package, so error patterns remain comparable across several packages. Small suffixes show what matters for troubleshooting without revealing the name: `-LANG` (longer than 15 characters), `-UML` (umlauts), `-SZ` (special characters).

The mapping stays **only in your database**. If someone gives you a placeholder (“step 2 fails on PC-017”), **Look up placeholder** finds the real name; **Download mapping (CSV)** returns the whole list (this is logged). Do not pass this file on.

> Software, package and collection names, exit codes and Windows error messages stay readable – they are needed for the analysis. If you use names of persons in package names or your own scripts, briefly check the package before passing it on (e.g. `fehler.txt`).

Security and Backup

- Sign-in only for members of the access group (including nested membership); lockout after repeated failed sign-ins.
- Web interface over HTTPS, cookies `HttpOnly` / `Secure` / `SameSite=Strict`, CSRF protection, strict Content Security Policy without external sources.
- Passwords (service account, repository account) are stored encrypted in the database; the key is `/data/pocketknife.key`.
- The service account password is passed as a parameter when steps run and does not end up in the clients' PowerShell logging.
- **Account lockout protection:** If AD rejects the service account, POCKET KNIFE pauses all automatic access until the cause is fixed ("Check & continue"). This applies to NTLM and Kerberos.
- **Failed sign-ins to the interface:** at most 8 per user and IP and 30 per IP within 15 minutes – spellings like `CORP\admin`, `x\admin` or `admin@corp.local` count together.
- **Sign-out when idle:** without mouse or keyboard input the session ends after 30 minutes (adjustable 5–720 minutes under Settings → Connection & Accounts; it cannot be turned off). Input in a remote session counts, automatic refreshes of open pages do not. A notice appears shortly before; open remote sessions are ended as well.
- **Changing the repository server:** whoever changes the server or share of the repository confirms this with the password of the access account – so the stored credentials never go to a mistyped or foreign server. Subfolders of the same share and mounted folders (`/repo`) need no confirmation.
- **Container without root:** the service runs as user `pocketknife` (UID 10001, `PK_UID` / `PK_GID`), with a read-only program file system, without capabilities and with `no-new-privileges`. On start, `/data` and `/backup` are handed over to it once. A mounted repository folder must be readable for it.
- **Pinned versions:** Python base image (version and digest), Debian packages and all Python packages are pinned exactly (see Update).
- **Active sessions** are checked against AD in the background every 2 minutes – even if the user is not clicking anything: anyone disabled or removed from the access or tenant group is signed out; anyone

additionally added to a tenant group or to the access group gets the extended scope; anyone who loses rights (e.g. a tenant) is signed out and signs in again – this also ends remote sessions. A brief AD disruption signs nobody out: the session stays and is checked again after one minute. OU admins, however, are signed out after at most 30 minutes without confirmation by AD. If the access group is missing in AD (deleted, renamed, ambiguous), nobody is a main admin until it is fixed (except the emergency access) – the log reports it. Signing out also ends open remote sessions.

- **LDAPS certificate:** each domain controller's certificate is stored on first contact and compared **before** a password is sent. A change (e.g. certificate renewal) is logged; with **strict check** (Settings → Connection & accounts) the connection is refused until confirmed. With `PK_LDAP_CA_FILE` (domain CA as PEM, e.g. `/data/ca.pem`) the certificate is fully validated against the CA. New installations have **strict check** enabled from 2.13; existing installations keep their setting – enabling it is recommended.
- **WinRM over HTTPS:** with `PK_WINRM_CA_FILE` (CA as PEM) the clients' certificate is validated; without this variable it is not validated, as before.
- **Request size:** requests to the web interface are limited to 2 MB (Explorer uploads 2 GB, winget packages 310 MB, certificates and package import 20 MB).
- **Sign-in error messages** do not reveal whether the user name or password was wrong or only the group membership is missing; details are in the log.
- **Working folder on the clients** (`C:\Windows\Temp\PocketKnife`): permissions for SYSTEM and Administrators only. If a user created the folder beforehand, it is moved aside and recreated (note in the log).
- **Repository as a folder** in the container only under `/repo` (`PK_REPO_MOUNT`) – the data directory cannot be set as repository.
- CSV exports neutralise values that Excel would execute as a formula.

Backup: POCKET KNIFE backs itself up – under **Settings → Backup**:

- **daily** at the configured hour (default 2:00 AM) and **before every update** (on the first start of a new version, before any database changes)

- **Versions to keep:** default 10; older backups are deleted automatically (including manual and “before update” ones)
- **Back up now**, list with download and delete
- Each ZIP contains: the database (consistent copy while running), `pocketknife.key` and the HTTPS certificate. This also restores stored passwords – so protect the backup folder.
- **Download** via the interface delivers the backup **without** `pocketknife.key` and the HTTPS key (database and key stay separate). After restoring from it, re-enter the service account, repository account and VNC password – or take the full ZIP directly from the backup folder on the Docker host.
- Location: `PK_BACKUP_PATH` from the `.env` (default `<PK_DATA_PATH>-backup`, e.g. `/opt/pocketknife/data-backup`). Preferably another drive or a mounted network share; if the folder is on the same drive as the database, the page points this out.

Restore: `docker compose down` → delete `pocketknife.db-wal` and `pocketknife.db-shm` in the data directory → extract the ZIP there → `docker compose up -d`.

Troubleshooting

Self-test: `docker compose exec pocketknife python3 tools/selftest.py` checks with a temporary database that previously fixed bugs have not returned. It does not change any data.

Message	Cause / Solution
LDAPS connection failed	Check the DC names, port 636 and DNS servers in the `.env`; is LDAPS enabled on the DC?
Service account locked / 0xC0000234	Unlock the account in AD, check the password, then click “Check & continue”
No access: not a member of the group	Add the user to the access group and sign in again
WinRM port not reachable	WinRM GPO, firewall, is the computer turned on?
Access denied	The service account is not a local administrator
Copy fails	Port 445 / ADMIN$ on the client, read access to the repository share
winget not found	Enable automatic installation or use the “Check / install winget” tool
Step as service account does not start (“Prozess als Dienstkonto konnte nicht gestartet werden”)	check the WMI service (Winmgmt) on the client, the service account must be a local administrator – or set the step to “Local System”
Custom command as service account cannot reach a share	No password is stored on the client (from 2.14). Have the file copied with “File from the repository” or run as “Local System”
Warning “/repo is not readable”	The service runs as UID 10001: on the Docker host `chmod -R o+rX` or `chown -R 10001` the repository folder
PowerShell scanner returns nothing	Scanner dialog → Test; the “PowerShell” profile needs a trigger
Remote session: guacd not reachable	Is the `pocketknife-guacd` container running? `docker compose up -d`

Glossary

Term	Explanation
ADMIN$	administrative share `C:\Windows` of a computer; POCKET KNIFE copies package files there
App Installer	Windows package that contains winget
Heartbeat	reachability check of all computers
LAPS	management of local administrator passwords through AD
LDAPS	encrypted access to Active Directory (port 636)
Product code	unique GUID of an MSI installation
Scan age	time since the last successful scan of a profile
WinRM	Windows Remote Management (PowerShell Remoting), port 5985/5986